Arturito
Rover autónomo diferencial mapeador

Autores

Joaquín Guzmán Legajo: 03751/4

Tomás Gamarra Legajo: 03852/8

Robaldi Santiago Legajo: 03769/4

Gonçalves Federico Legajo: 03866/5

Universidad Nacional de La Plata

Facultad de Ingeniería – Departamento de Electrotecnia

Asignatura: Taller de Proyecto I

5 de noviembre de 2026

Índice

1. Introducción **3**

2. Objetivos **4**
2.1. General . . . 4
2.2. Primarios . . . 4
2.3. Secundarios . . . 4
2.4. Diagrama en Bloques Funcionales . . . 5

3. Requerimientos **6**
3.1. Funcionales . . . 6
3.1.1. Bloque de Sensado . . . 6
3.1.2. Bloque de Comunicaciones . . . 7
3.1.3. Bloque de Tracción y Actuación . . . 8
3.1.4. Bloque de Procesamiento Central . . . 8
3.2. No Funcionales . . . 9
3.2.1. Rendimiento y Tiempos de Respuesta . . . 9
3.2.2. Precisión y Calidad del Mapa . . . 9
3.2.3. Seguridad y Robustez . . . 9
3.2.4. Autonomía y Energía . . . 9
3.2.5. Restricciones Físicas . . . 10

4. Diseño de hardware **11**
4.1. Diagrama de bloques funcionales y módulos de hardware . . . 11
4.2. Bloque de Sensado . . . 12
4.2.1. Sensor de distancia: módulo VL53L0X . . . 12
4.2.2. Sensor de ángulo de rueda: módulo AS5600 . . . 12
4.3. Bloque de Tracción y Actuación . . . 12
4.3.1. Componentes y justificación . . . 12
4.3.2. Características del motor GA12-N20 . . . 13
4.3.3. Condiciones de operación en el proyecto final . . . 13
4.4. Prototipado del subsistema de movimiento . . . 13
4.4.1. Objetivos y componentes . . . 13
4.4.2. Configuración inicial de alimentación (prototipo 1) . . . 14
4.4.3. Algoritmo de prueba . . . 14
4.4.4. Problemas encontrados y resolución . . . 15
4.4.5. Configuración final de alimentación (prototipo 2) . . . 15
4.4.6. Ensayos de consumo . . . 16
4.5. Alimentación del sistema y elección de la fuente . . . 17
4.5.1. Descarte de la pila de 9 V . . . 17
4.5.2. Consumos de cada bloque . . . 17
4.5.3. Arquitectura de alimentación . . . 17
4.5.4. Estimación de autonomía del ramal de lógica . . . 18
4.5.5. Estimación de autonomía del ramal de potencia . . . 18
4.5.6. Protección y carga de las celdas . . . 18
4.6. Interfaces eléctricas entre bloques . . . 18
4.6.1. Interfaz EDU-CIAA – DRV8833 . . . 18
4.6.2. Bus I^2C y salidas PWM de sensores . . . 19

4.7. Circuito esquemático 20
4.8. Diseño mecánico 20
4.8.1. Arquitectura general y selección de materiales 20
4.8.2. Requisitos del chasis 21
4.8.3. Tracción y apoyo 21
4.8.4. Integración de sensores 21
4.8.5. Estado de fabricación y próximos pasos 22

5. Modelado, Control y Algoritmos de Navegación 23
5.1. Bloque de Tracción y Actuación 23
5.1.1. Modelo dinámico e Identificación de la Planta del Motor GA12-N20 23
5.1.2. Lazo Interno: Diseño y Sintonización de Controlador PI del Motor GA12-N20 23
5.1.3. Análisis Discreto e Implementación Algorítmica 26
5.2. Bloque de Cinemática 27
5.2.1. Transformación de Marcos y Cinemática del Chasis 27
5.2.2. Algoritmos de Control Cinemático para Navegación 29
5.2.3. Análisis Discreto e Implementación Algorítmica 32
5.3. Bloque de Mapeo y Estimación de Posición 33
5.3.1. Análisis Experimental de Deriva Odométrica en theta 33
5.3.2. Corrección de orientación mediante IMU 33
5.3.3. Modelado del sensor TOF y construcción de la grilla de ocupación 33
5.4. Bloque de Navegación Autónoma 33
5.4.1. Estrategia de exploración y selección de metas 33
5.4.2. Algoritmo de evasión de obstáculos 33
5.5. Integración de Lazos y Arquitectura de Control Global 33
5.5.1. Diagrame unificado del sistema de control 33
5.5.2. Máquina de estados y lógica de selección de modos 33

6. Diseño de Firmware 34
6.1. Arquitectura propuesta y criterios de diseño 34
6.2. Las Tres Capas 35
6.3. Software de PC e interfaz de usuario prevista 37
6.4. Bibliotecas y herramientas externas previstas 38
6.5. Versiones y ensayos realizado 40
6.5.1. Pruebas IMU y ToF 40
6.5.2. Estructura de compilación y ejecución de aplicaciones 42
6.5.3. Sistema de telemetría 43
6.5.4. Pruebas de sensores y actuadores 44

7. Cronograma Preliminar 46

8. División de Tareas del Grupo 46

9. Bibliografía 46

10. Anexo 48
10.1. Circuito Esquemático 48
10.2. Lista de materiales (BOM) 50

1. Introducción

El problema que da origen a este proyecto es la necesidad de explorar, mapear y navegar dentro de áreas desconocidas por parte de robots. En entornos dinámicos como fábricas y hogares, los métodos tradicionales de guiado -como un robot seguidor de línea- requieren modificar el entorno agregando marcas físicas. Cuando estas modificaciones del entorno no son posibles, surge el problema de un robot que se adapte a su entorno, y no al revés.

Una posible solución es un robot controlado de manera manual por un teleoperador. Pero esto pronto encuentra sus limitaciones al tenerse en cuenta que se necesita de la constante atención de dicho teleoperador, un enlace inalámbrico ininterrumpido y no escala bien si se quiere mapear el espacio a explorar. Surge entonces la idea de un vehículo capaz de operar de manera autónoma, capaz de explorar, mapear y navegar un espacio desconocido por su cuenta. Pero para poder hacer esto posible, han de resolverse los siguientes problemas matemático-físicos en simultáneo: Percepción: ¿Qué elementos u obstáculos hay a mi alrededor?, Localización: ¿Dónde me encuentro dentro de ese espacio? y Planificación: ¿Cómo me muevo dentro de ese espacio?.

El paradigma de la industria: SLAM (Simultaneous Localization and Mapping). Bajo este paradigma se busca discretizar el espacio explorado a una grilla de ocupación donde se representan zonas libres, ocupadas e inexploradas. Esta grilla es generada con el uso de sensores de distancia con barrido angular. Para mitigar las imprecisiones se hace uso de sensores odométricos e inerciales.

La motivación de este proyecto surgió con la intención de poder implementar algún tipo de robot que tenga la capacidad de sensar información del entorno y, en base a ella, tomar decisiones en forma dinámica para cumplir con un objetivo específico. Originalmente se consideró realizar un robot seguidor de línea. Sin embargo, el hecho de que el robot esté limitado a seguir un recorrido marcado por una pista predefinida generó una desmotivación en cuanto al objetivo final a resolver. En base a esta primera idea, se disparó la posibilidad de hacer un robot que pueda desplazarse de manera autónoma esquivando cualquier obstáculo que se presente en el recorrido. Finalmente el alcance se amplió para que el propio robot pueda realizar un mapeo del entorno, escaneando los diferentes límites y obstáculos, para que una vez mapeado pueda navegarlo.

Por lo tanto, el presente proyecto consiste en realizar la implementación de un robot móvil diferencial autónomo con capacidad de mapear áreas y representarlas en algún formato 2D para su visualización en algún dispositivo, así como también para su posible recorrido evitando obstáculos, con un presupuesto acorde como proyecto institucional.

2. Objetivos

2.1. General

Diseñar e implementar un robot móvil autónomo con tracción diferencial capaz de explorar un entorno mediante sensores de distancia, mapearlo en 2D y transmitir la información en tiempo real.

2.2. Primarios

- **Plataforma y Actuación (Hardware):** Implementar la estructura mecánica del vehículo de tracción diferencial, la etapa de potencia (drivers) y los sensores odométricos para el registro del giro de las ruedas.
- **Sensado y Percepción:** Integrar un subsistema de medición de distancia para la detección de obstáculos y la caracterización del entorno.
- **Procesamiento Central y Control (Software):** Desarrollar el firmware para la integración cinemática, el control de velocidad en lazo abierto, la estimación de pose, la generación de la grilla de ocupación y el algoritmo de navegación autónoma.
- **Comunicaciones:** Establecer un enlace inalámbrico para la transmisión de datos de estado y del mapa generado hacia el dispositivo externo.
- **Visualización de la grilla de ocupación:** Desarrollar una interfaz gráfica (GUI) para la representación del mapa de ocupación en un dispositivo externo.

2.3. Secundarios

- **Planificación de Rutas:** Permitir desde un dispositivo remoto definir una ruta a seguir por el robot sobre el mapa de ocupación generado.
- **Control Modular:** Implementar la lógica de arbitraje de comandos para conmutar entre la navegación autónoma y el modo de teleoperación manual.
- **Optimización del Control de Velocidad**: Desarrollar un algoritmo de control de velocidad con realimentación a lazo cerrado (PID).
- **Optimización de Algoritmo para la generación de la Grilla de Ocupación**: Implementar algoritmo de tipo SLAM para la reconstrucción del entorno en una grilla de ocupación.

2.4. Diagrama en Bloques Funcionales

A continuación, en la Figura 1, se plantea una división del sistema en bloques que, cumpliendo distintas funcionalidades, colaboran en conjunto para lograr el objetivo de este proyecto.

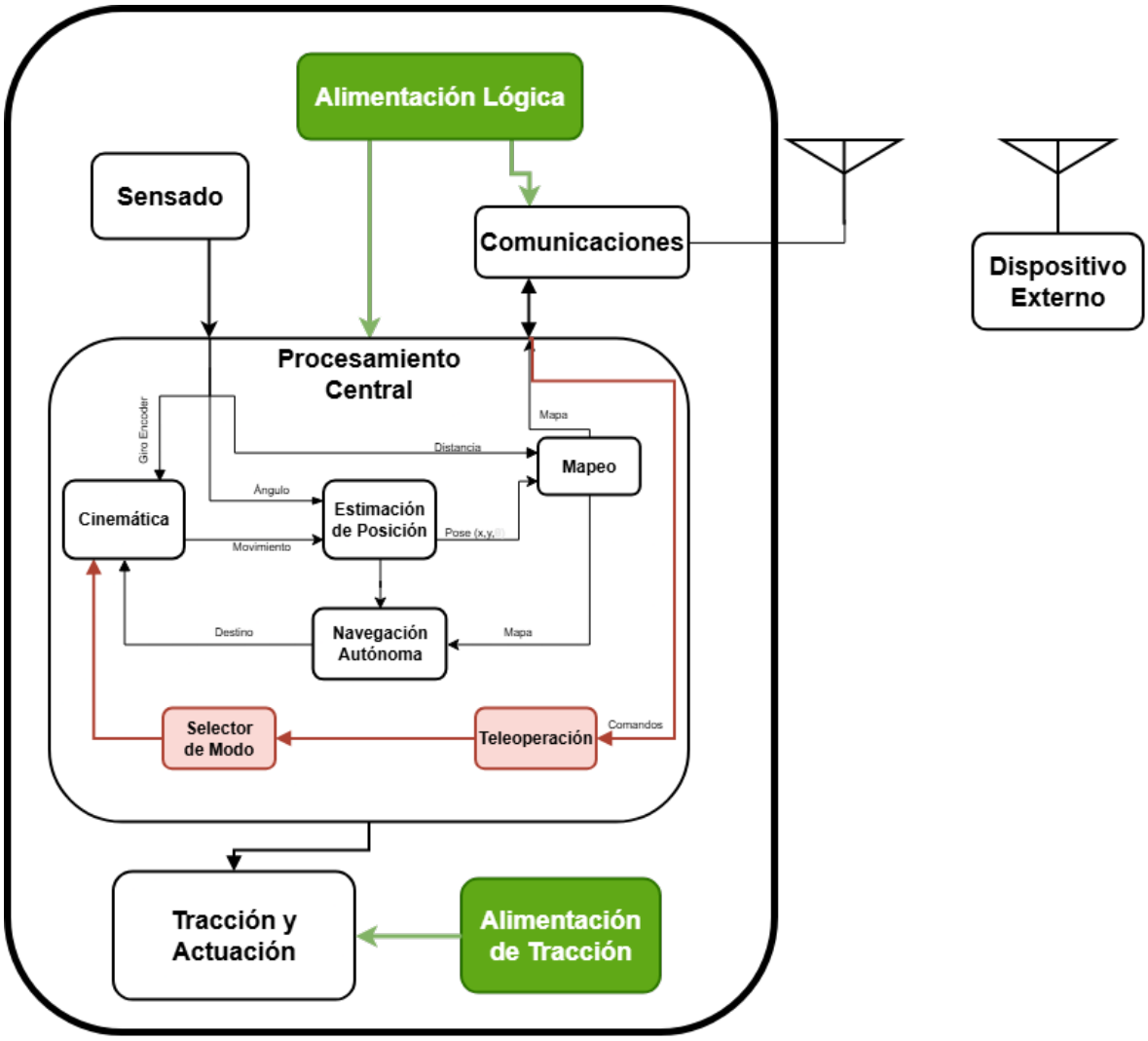


Figura 1: Esquema General del Sistema.

Nota: Los bloques resaltados en **rojo** dentro del diagrama corresponden a funcionalidades clasificadas como **objetivos secundarios** del proyecto.

- **Bloque de Sensado:** Se encarga de transmitir los datos primarios recolectados por sensores de distancia y sensores necesarios para la odometría al **Bloque de Procesamiento Central**.
- **Bloque de Comunicaciones:** Parte del sistema encargada de establecer un enlace con un dispositivo externo, para la transmisión del mapa generado en tiempo real y posición en el mismo.
- **Bloque de Tracción y Actuación:** Recibe por parte del **Bloque de Procesamiento Central** comandos para regular la velocidad deseada de las

ruedas, convirtiendo estas señales en tensión y corriente directas de la batería a los motores de dichas ruedas.

- **Bloque de Procesamiento Central:** Parte funcional del sistema encargada de: mapeo y estimación de posición, navegación y cinemática.
  - **Bloque de Cinemática:** Conoce la geometría del chasis (diámetro de ruedas, separación de ejes). Procesa los datos de sensores y da estimación de pose odométrica (deltas de desplazamientos en coordenadas y rotación del chasis/plataforma). Esta estimación primaria es consumida por el **Bloque de Mapeo y Estimación de Posición**.
  - **Bloque de Mapeo y Estimación de Posición:** Recibe la posición estimada por Odometría del **Bloque de Cinemática** para junto con los datos del sensor de distancia generar un mapa de ocupación del espacio 2D en el que el robot se mueve y dar una estimación final de su posición en el mismo. Tanto esta estimación final de la posición como el mapa son consumidos por el **Bloque de Navegación Autónoma**.
  - **Bloque de Navegación Autónoma:** Recibe el mapa y posición del robot en el mismo generados por el **Bloque de Mapeo y Estimación de Posición** y decide a dónde ha de moverse el robot. Estos parámetros de desplazamiento son enviados al **Bloque de Cinemática** para generar el desplazamiento correspondiente.
  - ***Bloque Selector de Modo:*** *Según el modo de navegación elegido desde el dispositivo externo, el robot navegará de manera autónoma o teleoperada.*
  - ***Bloque de Teleoperación:*** *Recibe desde el dispositivo externo las instrucciones de movimiento del robot.*
- **Dispositivo Externo:** Del **Bloque de Comunicaciones** recibe, procesa y muestra el mapa de ocupación junto con parámetros de estado del robot. Además permite entrada de comandos tales como: inicio y parada. *Objetivo secundario: Transmite comandos de selección de modos y, en caso de estar en modo de teleoperación, envía comandos de control de movimiento al robot.*

3. Requerimientos

3.1. Funcionales

3.1.1. Bloque de Sensado

Sensado de Distancia

- **Hardware:**
  - El sistema de sensado de distancia debe contar con un mecanismo de rotación que permita un barrido angular de 360°.
  - El mecanismo giratorio debe permitir determinar la posición angular del sensor de distancia para cada medida.
- **Software:**

- El software debe asociar en tiempo real cada lectura de distancia r con su respectivo ángulo de posición θ en el instante de muestreo.
- El software debe identificar medidas fuera del rango de validez $[R_{\min}, R_{\max}]$ o lecturas erróneas, sustituyéndolas por valores no válidos o no alcanzables.
- El software debe convertir las mediciones recibidas a unidades del Sistema Internacional (metros para la distancia y radianes para ángulo).
- El módulo debe agrupar el conjunto de mediciones (r, θ) correspondientes a un barrido completo en un único paquete de datos.
- El software debe transmitir el paquete de barrido procesado hacia el Bloque de Mapeo y Estimación de Posición.
- El software debe asignarle una estampa de tiempo a cada paquete al finalizar el barrido.

Sensado de Odometría (Velocidad de giro de ruedas y datos de IMU)

- **Hardware:**
  - El sistema de sensado de odometría debe poder medir aceleración lineal en 3 ejes a través de una IMU.
  - El sistema de sensado de odometría debe poder medir la cantidad de giros/pulsos en un intervalo de tiempo determinado que cada rueda realiza por separado mediante encoders.
- **Software:**
  - El software debe transformar las mediciones hechas por los sensores a unidades estándar del Sistema Internacional.
  - El software debe asignar una estampa de tiempo a cada paquete de lectura (IMU y encoders).
  - El software debe transmitir el paquete de lecturas hacia el Bloque de Cinemática.

3.1.2. Bloque de Comunicaciones

- **Hardware:**
  - El sistema debe contar con un módulo de transceptor inalámbrico para establecer la comunicación con el dispositivo externo mediante un protocolo estándar (ej. Wi-Fi / Bluetooth).
- **Software:**
  - El software debe poder recibir los datos del Bloque de Mapeo y Estimación de Posición (matriz de ocupación y pose del robot).
  - El software debe procesar comandos externos provenientes del dispositivo externo (START, STOP).

3.1.3. Bloque de Tracción y Actuación

- **Hardware:**
  - El sistema deberá contar con dos motores DC independientes para generar un movimiento diferencial.
  - El sistema deberá contar con una rueda loca (caster wheel) auxiliar para dar estabilidad mecánica al chasis sin limitar la maniobrabilidad.
  - El sistema deberá contar con un driver de potencia encargado de modular la energía provista por el subsistema de alimentación hacia los motores DC.

3.1.4. Bloque de Procesamiento Central

Bloque de Cinemática

- **Software:**
  - El software debe procesar los paquetes de datos provenientes del Bloque de Sensado (encoders e IMU) para resolver el modelo cinemático diferencial e integrar la pose odométrica.
  - El software debe transmitir la pose odométrica calculada hacia el Bloque de Mapeo y Estimación de Posición.
  - El software debe resolver las ecuaciones de movimiento para convertir las consignas en velocidades objetivo individuales para la rueda izquierda y derecha.
  - El software debe ejecutar el algoritmo de control en lazo abierto para ajustar la velocidad de cada rueda en base a las señales de Modulación por Ancho de Pulsos (PWM) y las señales lógicas de sentido de giro destinadas al Bloque de Tracción y Actuación.

Bloque de Mapeo y Estimación de Posición

- **Software:**
  - El software debe inicializar y gestionar una grilla de ocupación 2D discreta de dimensiones y resolución espacial predefinidas ($N \times M$).
  - El software debe transformar las coordenadas polares de cada barrido (r, θ) a coordenadas cartesianas globales (x, y) considerando la pose del robot.
  - El software debe implementar un algoritmo de trazado de rayos para actualizar el estado de la grilla de ocupación, identificando las celdas atravesadas como espacio libre y la celda de impacto como obstáculo.
  - El software debe calcular la probabilidad de ocupación de cada celda utilizando un modelo probabilístico incrementando la certeza ante múltiples mediciones consistentes.
  - El software debe transmitir la grilla de ocupación actualizada y la pose del robot al Bloque de Navegación Autónoma y al Bloque de Comunicaciones.

Bloque de Navegación Autónoma

- **Software:**
  - El software debe evaluar la matriz de ocupación recibida del Bloque de Mapeo y Estimación de Posición para identificar zonas no exploradas o caminos libres de obstáculos.
  - El software debe seleccionar el siguiente punto de destino o la dirección de avance libre dentro del mapa.
  - El software debe generar las consignas de velocidad lineal y angular requeridas para desplazar el robot hacia el destino seleccionado sin colisionar y enviarlas al Bloque de Cinemática.

3.2. No Funcionales

3.2.1. Rendimiento y Tiempos de Respuesta

- El Bloque de Mapeo y Estimación de Posición debe procesar y actualizar la matriz de ocupación a una tasa de al menos *[FreqMinMapeo]*.
- El Bloque de Comunicaciones debe enviar el estado del robot y el mapa al Dispositivo Externo a una tasa mínima de *[FreqMinComunicaciones]*.
- El software debe muestrear los contadores de las ruedas y las variables de la IMU a un intervalo de tiempo fijo $[T_s]$.

3.2.2. Precisión y Calidad del Mapa

- La grilla del mapa de ocupación debe tener una resolución mínima de *[ResolMinMapaOcup]*.
- El sistema debe ser capaz de representar un entorno cerrado de una dimensión mínima de *[DimMinMapaOcup]* y una dimensión máxima de *[DimMaxMapaOcup]*.
- El sistema debe detectar un objeto con una distancia mínima de *[DistMinObj]* y una distancia máxima de *[DistMaxObj]*.
- El hardware de sensado de distancia debe poder medir distancia a obstáculos en un rango $[R_{\min}]$ a $[R_{\max}]$ con una determinada precisión.

3.2.3. Seguridad y Robustez

- Ante una pérdida de enlace inalámbrico con el dispositivo externo superior a *[MaxSegSinConex]*, el Bloque de Navegación debe detener totalmente la marcha de la plataforma.
- El software debe gestionar la reconexión automática del enlace de comunicación en caso de pérdida temporal de señal.

3.2.4. Autonomía y Energía

- El sistema de alimentación debe proveer energía para que el sistema sea capaz de operar por al menos *[TiempoMinOperacion]*.

3.2.5. Restricciones Físicas

- El robot debe desplazarse a una velocidad lineal regulable entre *[VelocidadMin]* y *[VelocidadMax]*.
- El robot debe operar sobre pisos planos en interiores.
- El robot debe operar en un entorno donde los obstáculos tengan un mínimo de altura *[AlturaMinObstaculos]*.

4. Diseño de hardware

4.1. Diagrama de bloques funcionales y módulos de hardware

La Figura 2 muestra los módulos de hardware elegidos para cada bloque funcional definido en el TP1 y la forma de comunicación entre ellos. La Tabla 1 resume los componentes principales; su justificación se detalla en las secciones siguientes.

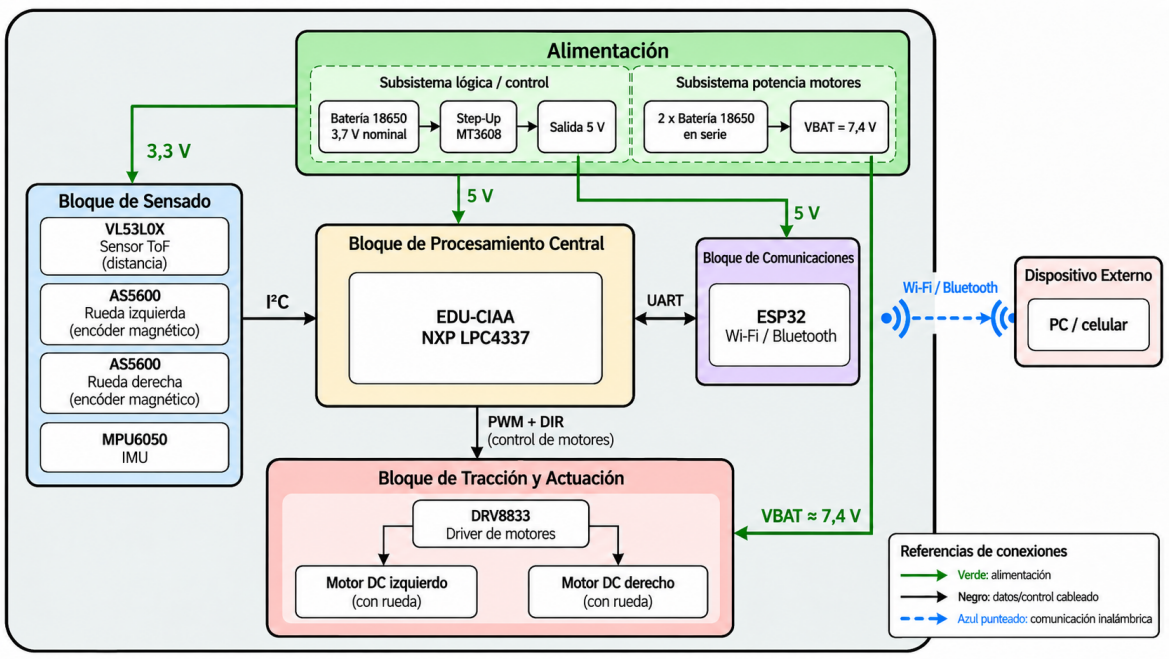


Figura 2: Módulos de hardware utilizados en cada bloque funcional y forma de comunicación.

Tabla 1: Resumen de componentes principales.

Componente	Código / valor	Formato físico	Bloque
Motorreductor DC	GA12-N20, 6 V–12 V, 150:1	Motor con caja metálica	Tracción
Puente H doble	DRV8833	Módulo	Tracción
Elevador de tensión	Módulo MT3608	Módulo	Alimentación
Celda Li-ion	18650, 3,7 V	Cilíndrica	Alimentación
Portapilas	Uno para 1 celda 18650 y uno para 2 celdas 18650 en serie	Portapilas	Alimentación
Microcontrolador de prototipado	Arduino Nano	Módulo	Prototipo
Módulo WiFi	ESP32 DevKit v1	Módulo	Comunicaciones
IMU	MPU6050	Módulo	Sensado
Sensor de distancia ToF	VL53L0X	Módulo	Sensado
Sensor magnético de ángulo	AS5600 (TinyTronics AS5600MOD, SKU 004634), x2	Módulo	Sensado
EDU-CIAA-NXP	LPC4337	Placa	Procesamiento

4.2. Bloque de Sensado

4.2.1. Sensor de distancia: módulo VL53L0X

La medición de distancia se realiza con un sensor de tiempo de vuelo (ToF) VL53L0X[8]. Está fijo sobre el chasis y mide distancias de hasta 2 m mediante un emisor láser de haz angosto, lo que lo hace adecuado para la detección precisa de obstáculos y permite resolver los límites del entorno con buena definición angular. El módulo puede alimentarse con 2,6 V a 5,5 V; su consumo según la hoja de datos es de 10 mA. Se comunica por I^2C y los proveedores informan un rango de medición de 30 mm a 2000 mm.

4.2.2. Sensor de ángulo de rueda: módulo AS5600

Para estimar el giro de cada rueda se utilizan dos módulos AS5600 [9], uno por rueda. El AS5600 es un sensor magnético de posición angular absoluta de 12 bits (4096 posiciones por vuelta) con salidas I^2C, PWM y analógica. El módulo seleccionado funciona por defecto a 3,3 V, tiene pines de paso 2,54 mm e incluye un imán de magnetización diametral. Los detalles de montaje del imán y del módulo se describen en la Sección 4.8.

4.3. Bloque de Tracción y Actuación

Este bloque debe contar con dos motores DC independientes para generar un movimiento diferencial, una rueda de apoyo auxiliar y un driver de potencia que module la energía provista por el subsistema de alimentación hacia los motores.

4.3.1. Componentes y justificación

- **Motores GA12-N20.** Se eligieron por: (i) su tensión nominal de 6 V a 12 V, que permite alimentarlos desde dos celdas 18650 en serie (7,4 V) a través del puente H; (ii) su baja velocidad con alto torque, gracias a la reducción de 150:1 con engranajes metálicos, adecuada para una plataforma que debe desplazarse lentamente y girar sobre su eje de manera controlada durante el escaneo; (iii) su bajo consumo; (iv) su tamaño y masa reducidos [10]. Sus características completas se resumen en la Tabla 2.
- **Puente H DRV8833.** Es un puente H doble basado en transistores MOSFET de bajo consumo. De su hoja de datos se destacan: (i) rango de tensión de alimentación de 2,7 V a 10,8 V, que cubre tanto una celda Li-ion (3,7 V) como dos celdas en serie (7,4 V); (ii) corriente de 1,5 A RMS y 2,0 A de pico por puente H (uno por motor), superior a la corriente de bloqueo del motor; (iii) protección térmica y contra baja tensión (UVLO); y (iv) entradas compatibles con lógica de 3,3 V y 5 V.

4.3.2. Características del motor GA12-N20

Parámetro	Valor
Tensión nominal	6 V–12 V
Velocidad en vacío	50 RPM a 3 V; 100 RPM a 6 V; 200 RPM a 12 V
Velocidad con carga	80 RPM
Torque nominal	2 kg cm
Torque de bloqueo	3,4 kg cm
Corriente nominal	0,07 A
Corriente de bloqueo	1 A
Relación de reducción	1:150
Engranajes	Metálicos
Dimensiones de la caja reductora	15 mm×12 mm×10 mm
Longitud total	34 mm
Eje de salida	3 mm de diámetro × 10 mm de largo
Masa	10 g

Tabla 2: Características del motor GA12-N20 (hoja de datos del fabricante, Handson Technology [10]).

4.3.3. Condiciones de operación en el proyecto final

En el prototipo los motores se alimentaron con aproximadamente 4 V. En el proyecto final, el puente H entregará 7,4 V (dos celdas en serie), pero los motores funcionarán con un ciclo de trabajo de aproximadamente el 50 %, de modo que la tensión media aplicada (≈3,7 V) coincide aproximadamente con la utilizada en las pruebas. En consecuencia, los consumos medidos en el prototipo son representativos del funcionamiento final (Sección 4.4.6).

La hoja de datos indica una variación lineal de la velocidad con la tensión (50 RPM por cada 3 V aproximadamente). Con 7,4 V y ciclo de trabajo máximo se estima una velocidad de ≈123 RPM; con ciclo de trabajo de ≈50 % (tensión media de 3,7 V) resulta ≈62 RPM, similar a las ≈68 RPM esperables en el prototipo a 4,08 V. Con ruedas de 44 mm de diámetro, la velocidad lineal es

$$v = \omega\, r = \frac{62 \cdot 2\pi}{60}\,\mathrm{rad\,s^{-1}} \times 0{,}022\,\mathrm{m} \approx 0{,}14\,\mathrm{m\,s^{-1}}$$

al 50 % de ciclo de trabajo, y de ≈0,28 m s^{-1} al máximo, lo que deja margen de regulación por PWM. La corriente nominal de 0,07 A por motor es coherente con los ≈15 mA por motor medidos en vacío en el prototipo, ya que la corriente nominal corresponde a la condición de torque nominal.

4.4. Prototipado del subsistema de movimiento

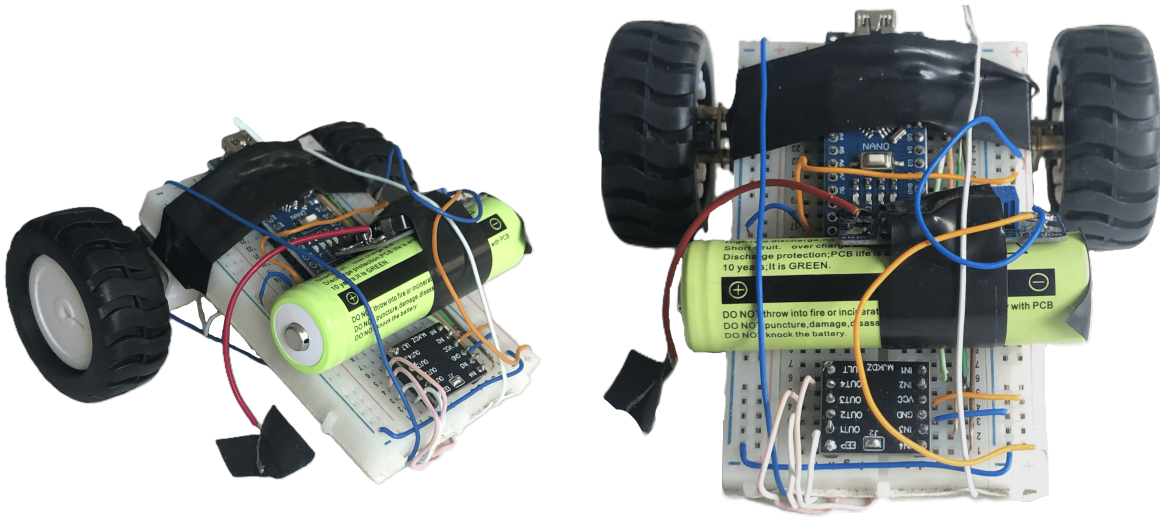
4.4.1. Objetivos y componentes

El prototipo se construyó con dos motores GA12-N20, un puente H DRV8833, un Arduino Nano como plataforma de prototipado para generar las señales de modulación

por ancho de pulso (PWM) y una celda 18650 de 3,7 V. El objetivo principal fue generar, desde el Arduino Nano, señales PWM hacia el puente H, de modo que éste aplique a cada motor la tensión requerida. Adicionalmente, se buscó adquirir conocimiento práctico sobre el comportamiento de los motores (fuerza, velocidad) y sobre el funcionamiento del DRV8833, con el fin de resolver con antelación los inconvenientes que pudieran surgir durante el ensamblaje del proyecto final.

4.4.2. Configuración inicial de alimentación (prototipo 1)

En una primera instancia, tanto el microcontrolador como los motores se alimentaron desde la misma celda de 3,7 V, a través de un regulador elevador (*step-up*) ajustado a 6 V. Esta tensión se aplicó al pin `VIN` del Arduino Nano y al pin `VCC` del DRV8833 (Figura 3).



(a) **(b)**

Figura 3: Prototipo 1: una única celda de 3,7 V con elevador de tensión a 6 V alimenta tanto al Arduino Nano como al puente H DRV8833.

4.4.3. Algoritmo de prueba

Todas las pruebas se realizaron con un mismo algoritmo, consistente en un lazo infinito que ejecuta la secuencia de la Tabla 3. El ciclo de trabajo (*duty cycle*) se definió como variable global, con el fin de evaluar el efecto del PWM sobre el comportamiento de los motores. La espera de 2 s entre acciones permite observar y medir el funcionamiento de los componentes. Para cada motor, el avance se logra con `IN1` = alto (PWM) e `IN2` = bajo, y el retroceso con `IN1` = bajo e `IN2` = alto (PWM).

Tabla 3: Secuencia de movimientos del algoritmo de prueba.

Etapa	Acción	Duración [s]
1	Espera	2
2	Avanzar	3
3	Espera	2
4	Retroceder	3
5	Espera	2
6	Girar (primer sentido)	3
7	Espera	2
8	Girar (sentido contrario)	3

4.4.4. Problemas encontrados y resolución

Pin de reposo (`nSLEEP`) del DRV8833. La primera prueba no resultó satisfactoria debido a que el DRV8833 dispone de un pin de reposo que debe mantenerse en nivel alto para que el controlador entregue tensión a los motores; de lo contrario, el dispositivo permanece en estado de bajo consumo (*sleep*). Dado que en esta etapa no se requería dicho modo, se utilizó el puente de soldadura (*jumper*) que provee el módulo para mantener este pin permanentemente en nivel alto. Una vez realizada esta corrección, el sistema funcionó correctamente.

Influencia de la tensión de alimentación. Se evaluó si el incremento de la tensión mediante el *step-up* mejoraba la velocidad y el torque de los motores. A partir del análisis de la hoja de datos y de las características de la fuente disponible, se concluyó que, para este proyecto, la prioridad es el torque y no la velocidad.

Alimentación del Arduino Nano a través del pin `VIN`. Durante las pruebas se alimentó el Arduino Nano con 5 V estables por el pin `VIN`. Dado que este pin se encuentra conectado a un regulador de tensión interno, la tensión efectiva de operación del microcontrolador resultó inferior a 5 V, lo que provocó un comportamiento irregular y la imposibilidad de enviar correctamente las señales al puente H. Según la hoja de datos del Arduino Nano, la alimentación por `VIN` debe encontrarse entre 6,5 V y 12 V. Este resultado constituye una consideración relevante para el ensamblaje final: al alimentar la EDU-CIAA y el ESP32, deberá verificarse para cada pin de alimentación si existe un regulador intermedio, y definirse en consecuencia la tensión a suministrar.

4.4.5. Configuración final de alimentación (prototipo 2)

Con base en lo anterior, se optó por separar la alimentación de la parte lógica y la de potencia (motores), como muestra la Figura 4:

- **Lógica:** el Arduino Nano se alimentó por el pin `VIN` con una pila de 9 V. La tensión en la alimentación del microcontrolador resultó de aproximadamente 7 V, atribuible presumiblemente a la resistencia interna de la pila. Esta tensión permite que los pines de salida alcancen cómodamente el nivel alto de 5 V.
- **Potencia:** la celda de litio, cargada a 4,12 V, alimentó el DRV8833 con 4,1 V, de modo que cada motor recibió 4,08 V al máximo ciclo de trabajo.

La medición de 7,1 V se realizó entre el pin `VIN` y `GND` del Arduino Nano, es decir, en la entrada del regulador lineal interno, antes de la regulación. La caída desde 9 V se debe a la resistencia interna de la pila bajo carga (*voltage sag*). A pesar de ello, la salida regulada (pin `5V`) entregó 5,0 V, dado que 7,1 V supera el umbral mínimo de entrada de 6,5 V.

Con esta configuración el prototipo funcionó de manera estable y eficiente.

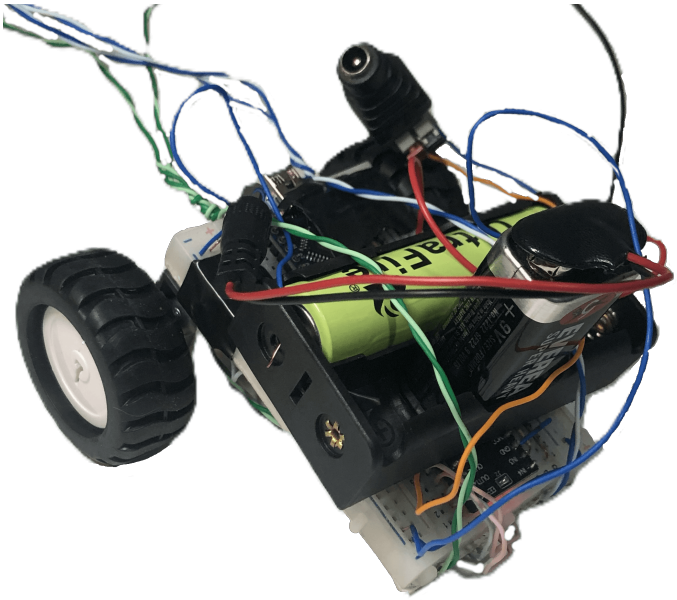


Figura 4: Prototipo 2: el Arduino Nano se alimenta con una pila de 9 V y el ramal de potencia (DRV8833 y motores) con una celda de 3,7 V cargada a 4 V.

4.4.6. Ensayos de consumo

Se midieron la tensión y la corriente consumida por el prototipo con el objetivo de estimar el consumo del sistema final. La Tabla 4 resume la corriente demandada por los **dos motores en conjunto** desde la celda de litio (en vacío, aproximadamente 15 mA por motor). El Arduino Nano, alimentado con la pila de 9 V, consumió entre 19 mA y 25 mA.

Tabla 4: Consumo de corriente de ambos motores en simultáneo (alimentación 4,1 V).

Condición de carga	Corriente [mA]
En vacío (sin carga)	30
Ligeramente frenados	60
Ruedas frenadas, sin llegar a inmovilizarse (máximo)	200

Como se explicó en la Sección 4.3.3, estos valores son representativos del funcionamiento final. Con modulación PWM, la corriente media del motor es aproximadamente la misma que con la tensión continua equivalente, mientras que la corriente tomada de la batería es del orden del ciclo de trabajo por la corriente del motor ($I_{\text{bat}} \approx D \cdot I_{\text{motor}}$), ya que la potencia se conserva ($3{,}7\,\text{V} \cdot I_{\text{motor}} \approx 7{,}4\,\text{V} \cdot I_{\text{bat}}$). Con $D \approx 0{,}5$, los 30 mA, 60 mA y 200 mA de la Tabla 4 equivalen a ≈15 mA, 30 mA y 100 mA tomados de las celdas.

4.5. Alimentación del sistema y elección de la fuente

4.5.1. Descarte de la pila de 9 V

A partir de los resultados anteriores, y considerando las hojas de datos de la EDU-CIAA y del ESP32 con comunicación WiFi, se concluyó que la pila de 9 V no resulta adecuada para el proyecto final, ya que presenta desgaste y caída de tensión durante la descarga.

Se utilizó una pila alcalina de 9 V en formato PP3 (6F22/6LR61). Su capacidad nominal es de 400 mAh a 500 mAh con corrientes muy bajas, pero debido a su alta resistencia interna, con consumos continuos superiores a 50 mA su capacidad útil se reduce a aproximadamente 250 mAh–350 mAh. Con un consumo conjunto de la EDU-CIAA y el ESP32 de 350 mA–550 mA, la autonomía resultaría de entre $250/550 \approx 0{,}45$ h y $350/350 = 1$ h.

4.5.2. Consumos de cada bloque

La Tabla 5 presenta el balance de consumo estimado del sistema final, con dos ramales de alimentación: lógica (referida a 5 V) y potencia (motores).

Tabla 5: Balance de consumo estimado del sistema final.

Bloque	Ramal	Tensión [V]	$I_{\text{mín}}$ [mA]	$I_{\text{máx}}$ [mA]
EDU-CIAA-NXP (LPC4337)	Lógica	5	180	320
ESP32 DevKit v1 (WiFi activo)	Lógica	5	160	240
Módulo VL53L0X	Lógica	3,3	10	20
Módulos AS5600 (x2)	Lógica	3,3	26	26
IMU (MPU6050)	Lógica	3,3	5	5
Total ramal de lógica (referido a 5 V)			**381**	**611**
Motores GA12-N20 (x2), corriente de batería	Potencia	7,4 (PWM ≈50 %)	30	200

4.5.3. Arquitectura de alimentación

El sistema se alimenta mediante dos ramales independientes con masa común:

- **Ramal de potencia (puente H DRV8833 y motores):** un portapilas con dos celdas 18650 en serie (7,4 V nominales, hasta 8,4 V cargadas), dentro del rango de alimentación del DRV8833 (2,7 V–10,8 V). La velocidad de los motores se regula por PWM, con un ciclo de trabajo de operación del orden del 50 %.
- **Ramal de lógica (EDU-CIAA, ESP32 y sensores):** una única celda 18650 de 3,7 V y un módulo elevador MT3608 ajustado a 5 V, que mantiene una tensión estable independientemente de la descarga de la celda. La salida de 5 V alimenta el pin de 5 V de la EDU-CIAA y el pin `5V/VIN` del ESP32 DevKit, delegando la conversión a 3,3 V en los reguladores internos de cada placa. Los sensores se alimentan a 3,3 V desde las placas.

4.5.4. Estimación de autonomía del ramal de lógica

Con una celda de 2200 mAh, la energía almacenada es

$$E_{\text{bat}} = 2{,}2\,\text{Ah} \times 3{,}7\,\text{V} \approx 8{,}14\,\text{Wh}.$$

La potencia consumida a 5 V (Tabla 5) y la potencia tomada de la celda, suponiendo una eficiencia del elevador $\eta = 0{,}85$, son

$$P_{\text{mín}} = 5\,\text{V} \times 0{,}368\,\text{A} = 1{,}84\,\text{W} \;\Rightarrow\; P_{\text{bat}} \approx 2{,}17\,\text{W},$$
$$P_{\text{máx}} = 5\,\text{V} \times 0{,}598\,\text{A} \approx 2{,}99\,\text{W} \;\Rightarrow\; P_{\text{bat}} \approx 3{,}52\,\text{W}.$$

La autonomía resulta entonces

$$t_{\text{máx}} = \frac{8{,}14\,\text{Wh}}{2{,}17\,\text{W}} \approx 3{,}8\,\text{h}, \qquad t_{\text{mín}} = \frac{8{,}14\,\text{Wh}}{3{,}52\,\text{W}} \approx 2{,}3\,\text{h},$$

con una corriente tomada de la celda de 0,59 A a 0,95 A.

Justificación. El valor de 2,3 h corresponde al caso en que todas las placas y sensores consumen simultáneamente su valor máximo de forma sostenida. Estos valores máximos son picos (por ejemplo, las ráfagas de transmisión WiFi del ESP32 y la actividad plena de los periféricos de la EDU-CIAA), por lo que en funcionamiento normal el consumo medio será inferior y se ubicará más cerca del extremo inferior, con lo que la autonomía real será mayor. Aun descontando un 30 % de la capacidad nominal por la capacidad útil de la celda, el peor caso es de ≈1,6 h. Para un proyecto de carácter académico, en el que las pruebas, las demostraciones y las corridas de mapeo duran del orden de minutos y la celda puede recargarse o reemplazarse entre sesiones, esta autonomía resulta suficiente.

4.5.5. Estimación de autonomía del ramal de potencia

Con dos celdas en serie (≈16,3 Wh) y una corriente de batería de 100 mA a 7,4 V (≈0,74 W), la autonomía del ramal de potencia es de ≈22 h. En el peor caso, con los motores bloqueados de manera continua y una corriente de batería de 0,62 A (≈4,6 W), la autonomía sería de ≈3,5 h. En ambos casos el ramal de potencia no es el factor limitante frente al ramal de lógica.

4.5.6. Protección y carga de las celdas

Aún no se definieron la protección de las celdas ni el cargador. **[PENDIENTE: definir el sistema de protección y el cargador, y reflejarlos en la lista de materiales.]**

4.6. Interfaces eléctricas entre bloques

4.6.1. Interfaz EDU-CIAA – DRV8833

El DRV8833 se controla mediante cuatro entradas lógicas (`IN1`–`IN4`), dos por cada puente H (uno por motor), compatibles con la lógica de 3,3 V de la EDU-CIAA. Los pines utilizados para cada entrada son GPIO18, LCD1, LCD2 y LCD3, respectivamente, como se muestra en el esquemático del sistema (Figura 21) y en el del poncho (Figura 22).

Tabla 6: Control de un motor mediante las entradas del puente H.

Acción	`IN1`	`IN2`
Avance	Alto (PWM)	Bajo
Retroceso	Bajo	Alto (PWM)

4.6.2. Bus I²C y salidas PWM de sensores

El módulo VL53L0X y la IMU MPU6050 se comunican por I²C (Tabla 7). Dado que los dos módulos AS5600 comparten la misma dirección fija (`0x36`) y esta no se puede reprogramar, no es posible conectarlos al mismo bus. Para resolverlo se investigó la hoja de datos del LPC4337, donde se halló que el microcontrolador dispone de dos interfaces I²C, aunque la placa EDU-CIAA-NXP solo expone una de ellas. Sin embargo, al revisar el esquemático para verificar si los pines dedicados de la segunda interfaz estaban accesibles, se encontró que los pines que la placa expone para la comunicación RS232 están conectados directamente a los pines del microcontrolador correspondientes a la segunda interfaz I²C.

Pruebas de validación. Se realizaron las pruebas mencionadas, conectando cada encoder a un bus I²C distinto y operando junto con el bloque de tracción, con el fin de evaluar si las mediciones eran correctas. Los resultados fueron satisfactorios y confirmaron que utilizar los dos buses I²C de la EDU-CIAA-NXP es una solución adecuada para nuestro hardware, ya que resuelve el problema de los dos encoders con la misma dirección fija.

El único inconveniente fue el peor desempeño del bus cuyos pines no están expuestos de forma nativa en la EDU-CIAA-NXP: algunas lecturas del encoder conectado a este bus fallaban o la comunicación no se establecía correctamente. Al revisar el esquemático de la placa se notó la ausencia de resistencias de pull-up en esos pines. A modo de prueba, se incorporaron resistencias de pull-up externas de 4,4 kΩ en este bus. Tras repetir las pruebas, ambos encoders funcionaron correctamente, sin pérdida de información ni fallos de comunicación.

Dado el resultado obtenido, no fue necesario recurrir a la alternativa de leer los encoders mediante la salida PWM (pin OUT) con *input capture*.

Tabla 7: Dispositivos conectados al bus I²C.

Dispositivo	Dirección (7 bits)	Observación
Módulo VL53L0X	0x29	Una única unidad; el módulo expone solo 4 pines
IMU MPU6050	0x68 / 0x69	Seleccionable con el pin `AD0`

4.7. Circuito esquemático

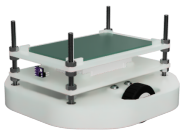
Criterios de modelado en KiCad. Los módulos integrados (ESP32, DRV8833 y MT3608) se modelan como cajas negras mediante tiras de pines (*pin headers*) de paso 2,54 mm, conectando únicamente sus entradas y salidas.
Componentes discretos de filtrado.

- Un capacitor electrolítico de 470 µF/16 V en paralelo con la alimentación de potencia del DRV8833 (pin `VM` a `GND`).
- Un capacitor cerámico de 100 nF (código 104), soldado directamente en los bornes de cada motor, para filtrar el ruido de conmutación (EMI).

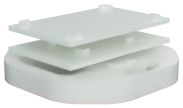
El circuito esquemático está presente en el anexo como la figura 21 y el esquemático del pongo, en la figura 22)

[PENDIENTE: Evaluar la adición de los componentes discretos de filtrado. Insertar el esquemático completo realizado en KiCad.]

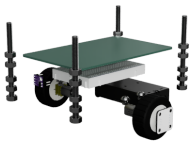
4.8. Diseño mecánico



(a) Modelo completo.



(b) Piezas impresas (ABS).



(c) Componentes/Hardware.

Figura 5: Desglose y vistas del modelo CAD paramétrico del chasis.

4.8.1. Arquitectura general y selección de materiales

El chasis del robot se concibió bajo una filosofía de diseño modular y apilable en niveles, modelado de forma paramétrica en Autodesk Fusion. **[PENDIENTE: agregar el modelo CAD en un anexo o en la bibliografía.]**

- **Proceso de manufactura:** impresión 3D por Fabricación por Filamento Fundido (FFF) en una impresora Creality Ender-3 S1.
- **Material:** ABS, por su resistencia mecánica y al impacto.
- **Unión de niveles:** la vinculación estructural vertical entre plataformas se resuelve mediante varillas roscadas y tuercas estandarizadas M3.

4.8.2. Requisitos del chasis

[PENDIENTE: incorporar estos requisitos a la sección de requerimientos del TP1 si aún no figuran.]

- **Sensores de ángulo:** cada módulo AS5600 debe quedar enfrentado a una rueda, con el imán de magnetización diametral pegado en el eje de la rueda y alineado con su eje de rotación.
- **Radio de giro:** el chasis debe permitir un radio de giro nulo (girar sobre su propio eje).
- **Sensor ToF:** el VL53L0X debe apuntar al frente, sobre el eje de simetría del chasis, con el campo de visión despejado durante el giro del robot.
- **Tracción:** el chasis debe poder desplazarse con los motores y ruedas disponibles.
- **Alimentación:** el chasis debe tener espacio para los dos sistemas de alimentación separados: un portapilas de una celda 18650 (lógica) y uno de dos celdas 18650 (potencia).

4.8.3. Tracción y apoyo

El sistema adopta una configuración cinemática de tracción diferencial, con dos motores GA12-N20 acoplados a ruedas de 44 mm de diámetro. El portapilas de dos celdas se ubica en la sección posterior, para desplazar el centro de gravedad hacia el punto de apoyo. En la versión CAD actual el apoyo trasero aún no está diseñado: será un patín deslizante integrado e impreso en ABS que se ensayará en la siguiente fase de prototipado; si las pérdidas por fricción resultan elevadas, se lo reemplazará por una rueda de bolas de acero (*ball caster*).

4.8.4. Integración de sensores

- **Módulos AS5600:** el chasis integra soportes enfrentados a cada rueda, con los ejes de rotación alineados. El imán se monta centrado en el extremo del eje de la rueda, con una separación respecto del sensor de entre 1 mm y 2 mm. Como el motor GA12-N20 no tiene eje trasero accesible, el imán no puede montarse en el rotor y gira a la velocidad de la rueda ($\approx$62 RPM), lo que simplifica el conteo de vueltas.
- **Soporte del sensor ToF:** se ubica en el extremo frontal, sobre el eje de simetría del robot, para garantizar un campo de visión despejado. Su geometría se encuentra en etapa de finalización.

La cobertura de 360° se obtiene haciendo girar el robot completo, por lo que el ángulo de cada medición proviene de la odometría (AS5600) y de la IMU. Conviene medir el desplazamiento del sensor ToF respecto del centro de giro (punto medio entre ruedas) para corregirlo en el cálculo del mapa. Dado que el robot gira apoyado en el patín, pueden producirse deslizamientos que degraden la estimación del ángulo solo con los

AS5600; el giróscopo de la IMU permite compensarlo.
La resolución angular depende de la velocidad de giro ω y del tiempo de medición T_s del sensor: $\Delta\theta = \omega\, T_s$. Con $T_s \approx 33\,\text{ms}$ y una muestra por grado, la velocidad máxima de giro es de $\approx 30\,^\circ/\text{s}$ (una vuelta completa en $\approx 12\,\text{s}$).

4.8.5. Estado de fabricación y próximos pasos

A la fecha de este informe, el diseño 3D se encuentra en fase de cierre en entorno CAD y aún no se inició la impresión de las piezas. Los próximos pasos son:

- **Pruebas de tolerancia:** impresión de piezas de prueba con distintas tolerancias para validar los encastres de las tuercas y varillas M3 y M6 antes de imprimir el chasis completo.
- **Impresión 3D e integración física:** Fabricación de los niveles y montaje de la electrónica.
- **Validación de Apoyo:** Evaluación experimental de la fricción del apoyo de ABS.

5. Modelado, Control y Algoritmos de Navegación

5.1. Bloque de Tracción y Actuación

5.1.1. Modelo dinámico e Identificación de la Planta del Motor GA12-N20

Desde el punto de vista físico, el motor DC (GA12-N20) no responde de forma instantánea a los cambios de tensión debido a la inercia del rotor y la fricción de los engranajes. A su vez, existe una relación proporcional entre el ciclo de trabajo de la señal PWM aplicada a través del puente H y la velocidad angular alcanzada en estado permanente.

Para representar este comportamiento dinámico en el dominio de Laplace, la planta se modela como un sistema de primer orden:

$$G(s) = \frac{\Omega(s)}{U(s)} = \frac{K}{\tau s + 1}$$

Donde

- $\Omega(s)$ es la velocidad angular de salida de la rueda, expresada en $[rad/s]$.
- $U(s)$ es la señal de control aplicada al puente H, expresada en unidades de PWM (Duty Cycle $[0 - 100\,\%]$)
- K es la ganancia del sistema, expresada en $[(rad/s)/unidadPWM]$.
- τ es la constante de tiempo mecánica, expresada en milisegundos $[ms]$, que representa el tiempo necesario para que la respuesta alcance el $63,2\,\%$ de su valor de estado permanente.

Para obtener los valores de K y τ para cada rueda (izquierda y derecha) se aplica un escalón de tensión partiendo del reposo y se registra la velocidad angular en función del tiempo mediante el sensor magnético AS5600.

Identificación de la Ganancia K Una vez alcanzado el régimen permanente, se calcula K como el cociente entre la velocidad angular alcanzada y el ciclo de trabajo definido por el escalón en U(s).

Identificación de la constante de tiempo τ Se determina identificando el instante de tiempo donde $t = \tau$ tal que la velocidad angular alcanzada sea del $63,2\,\%$.

5.1.2. Lazo Interno: Diseño y Sintonización de Controlador PI del Motor GA12-N20

Diseño en el Dominio Continuo
Habiendo identificado la función de transferencia de la planta (motor), es posible ahora plantear un lazo cerrado por PID de la velocidad angular de un motor, y por lo tanto de la rueda correspondiente.

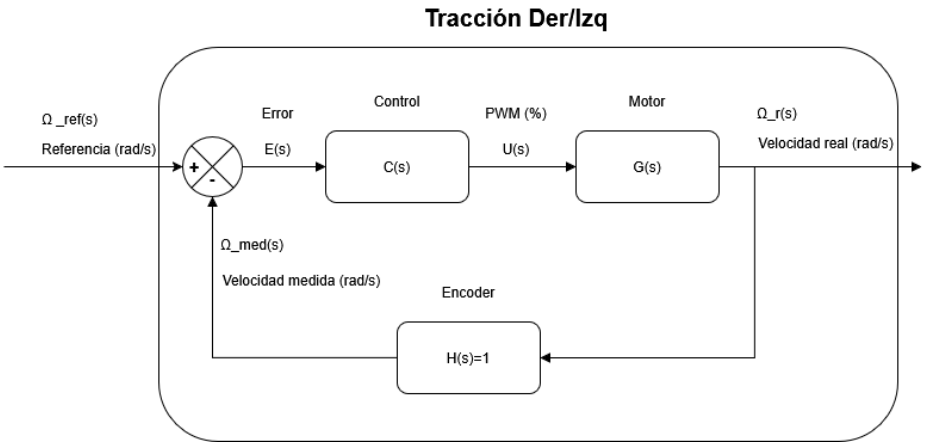


Figura 6: Lazo interno de control de un motor.

Donde:

- Ω_ref(s) es la velocidad angular de referencia para el motor.
- $E(s)$ es el error entre la velocidad de referencia y la velocidad medida.
- $C(s)$ es el control proporcional e integral.
- $U(s)$ es la señal PWM enviada al motor por $C(s)$.
- $G(s)$ es la función de transferencia del motor.
- Ω_r(s) es la velocidad angular real que tiene el motor.
- $H(s) = 1$ es la función de transferencia del encoder, se la asume sin retrasos.
- Ω_med(s) es la velocidad medida por el encoder, utilizada para el cálculo del error.

Siendo que la función de transferencia de la planta es

$$G(s) = \frac{K}{\tau s + 1}$$

Es posible plantear un control por PI

$$C(s) = K_p + K_i/s = K_p(\frac{s + K_i/K_p}{s})$$

Donde la función de transferencia a lazo abierto es (renombrando τ como τ_{la}, constante de tiempo de lazo abierto)

$$LA(s) = C(s)G(s) = K_p(\frac{s + K_i/K_p}{s})\frac{K}{\tau_{la}s + 1}$$

Analizando la función de lazo abierto, vemos es que posible cancelar uno de sus polos, eligiendo K_i y K_p tal que

$$\frac{K_i}{K_p} = \frac{1}{\tau_{la}} => K_i = \frac{K_p}{\tau_{la}}$$

Queda entonces la función de transferencia a lazo abierto

$$LA(s) = \frac{K_p K}{s\tau_{la}}$$

Al cerrar el lazo, la función de transferencia a lazo cerrado queda

$$LC(s) = \frac{LA(s)}{1 + LA(s)H(s)} = \frac{\frac{K_p K}{s\tau_{la}}}{1 + \frac{K_p K}{s\tau_{la}}1}$$

$$LC(s) = \frac{1}{\frac{1}{K_p K}s + 1} = \frac{1}{\tau_{lc}s + 1}$$

Donde $\tau_{;c}$ es la constante de tiempo a lazo cerrado. Siendo que depende del valor de la constante K_p, nos da un grado de libertad para definir el tiempo de respuesta que queremos que tenga el lazo cerrado de control.

$$\tau(K_p) = \frac{1}{K_p K}$$

$$K_i(K_p) = \frac{K_p}{\tau}$$

Desaturación proporcional

Se define ω_{max} como la velocidad angular máxima que admite cada rueda, se establece al 85 % de la velocidad angular máxima alcanzable por los motores bajo la condición de tensión mínima operativa de la batería (V_{batmin}). Este margen del 15 % permite a los controladores PI combatir perturbaciones sin saturar el ciclo de trabajo del PWM.

Cuando las velocidades de referencia dadas exceden ω_{max} se aplica un escalamiento proporcional $S = \frac{max(|\omega_{izq}|,|\omega_{der}|)}{\omega_{max}}$. Al dividir ambas referencias por S, se mantiene la relación $\frac{\omega_{der}}{\omega_{izq}}$, garantizando de esta forma la invariancia del radio de giro instantáneo R, previniendo así la deformación de la trayectoria.

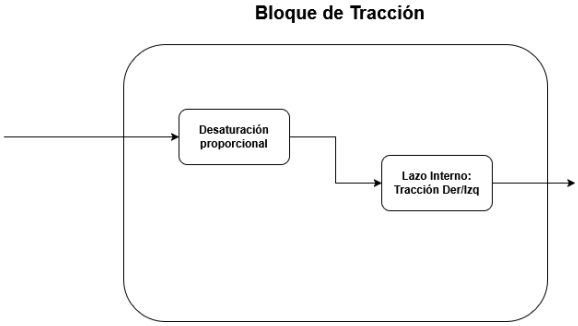


Figura 7: Flujo desaturación proporcional.

Acondicionamiento de la aceleración
Evaluar si conviene implementarlo

5.1.3. Análisis Discreto e Implementación Algorítmica

Frecuencia de funcionamiento
En el mundo real, es imposible integrar cada instante de tiempo. El intervalo de muestreo del estado del lazo cerrado (medición del sensor y computo de la lógica en consecuencia) se deberá de hacer cada cierto Δt. Para garantizar que el controlador digital represente fielmente el comportamiento diseñado en el dominio continuo, el intervalo de muestreo Δt debe ser lo suficientemente chico respecto a la velocidad de respuesta del lazo cerrado (τ_{lc}).

Un período de muestreo del orden de la constante de tiempo ($\Delta t \approx \tau_{lc}$) resulta inaceptable por: Dado que $t = \tau_{lc}$ la velocidad angular ya ha alcanzado el $63,2\,\%$ de su estado permanente para cuando se hace el primer muestreo. Esto impide detectar las variaciones bruscas de velocidad a lo largo del recorrido hacia el regimen permanente. Por otro lado, la aproximación de la integral por suma de rectángulos $e[k]\Delta t$ asume que el error varía suavemente entre muestras. Un Δt elevado induce error por discretización sustancial.

Por lo tanto, se toma el criterio empírico de diseño, el cual establece que se deben tomar al menos entre 5 y 10 muestras durante la constante de lazo cerrado:

$$\Delta t <= \frac{\tau_{lc}}{10}$$

Dado que la constante de tiempo del motor a lazo cerrado es de $\tau_{lc} \approx LLENAR$, se toma entonces un valor de $\Delta t = LLENAR$. Significando esto una frecuencia de funcionamiento del lazo de $f_m = 1/\Delta t$.

Consideraciones para la Implementación Algorítmica
Para trasladar el lazo de control diseñado en el dominio continuo s al microcontrolador, se requiere discretizar la ecuación diferencial en el dominio del tiempo t:

$$u(t) = K_p e(t) + K_i \int_0^t e(m)\, dm$$

Evaluando $t = k\Delta t$ (donde k representa el número de muestra actual y Δt el período de muestreo elegido anteriormente), se obtiene la ecuación en diferencias:

$$u[k] = K_p e[k] + \sum_{i=1}^{k} e[i]\Delta t$$

Donde la integral se aproxima mediante una suma de Riemann (rectángulos de ancho Δt y altura $e[k]$).

El cálculo directo de la sumatoria $\sum_{i=1}^{k} e[i]\Delta t$ requeriría de almacenar todo el historial de errores en memoria RAM, lo que incrementaría el tiempo de procesamiento a medida que el robot opera.

Para resolver esto, es posible expresar la integral k-ésima de manera recursiva, en función del cálculo anterior. Siendo $I[k]$ el término integral:

$$I[k] = I[k-1] + e[k]\Delta t$$

De esta forma el algoritmo solo conserva en memoria un único valor, resultando en la ecuación en diferencias final:

$$u[k] = K_p e[k] + K_i I[k]$$

[PENDIENTE: Mencionar en algún lado o hacer el análisis de que la salida del controlador PI (PWM) debería ser Max 100 %.]

5.2. Bloque de Cinemática

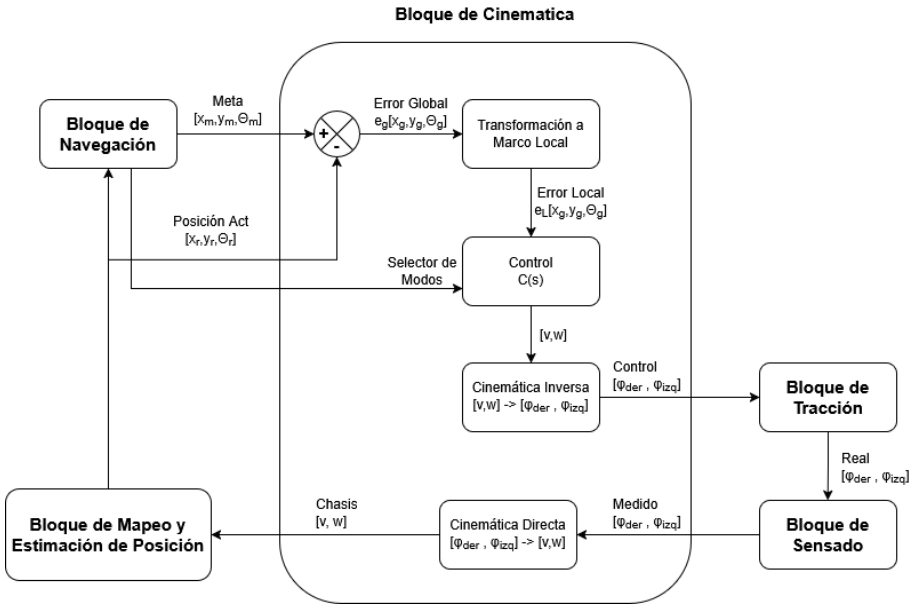


Figura 8: Diagrama general de flujo del bloque de cinemática.

La arquitectura general del Bloque de Cinemática es la mostrada en la figura de arriba.

5.2.1. Transformación de Marcos y Cinemática del Chasis

Marcos de Referencia y Transformaciones
Se plantean dos marcos de referencia, uno global, para mapeo y planificación de ruta, y otro local al robot.

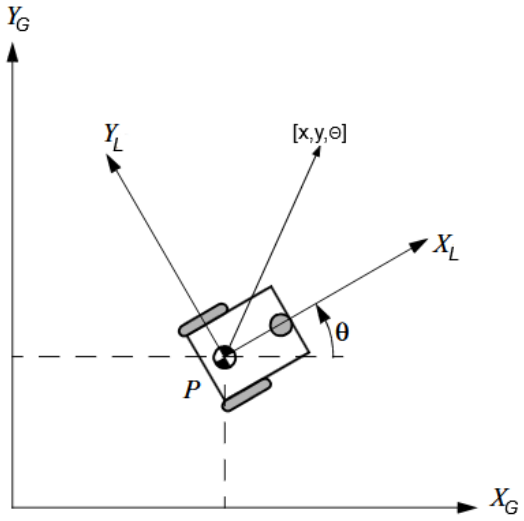


Figura 9: Marcos Global y Local.

Sea $[x, y, \theta]$ un punto cualquiera (un punto en el plano 2D con una orientación θ asociada) y $[x_p, y_p, \theta_p]$ las coordenadas del centro del robot respecto al marco global, entonces el vector de desplazamiento del centro del robot al punto es $[\Delta x, \Delta y, \Delta\theta]$. Este vector de error/desplazamiento puede traducirse del marco de referencia global al marco local:

$$\begin{bmatrix} \Delta x_L \\ \Delta y_L \\ \Delta\theta_L \end{bmatrix} = \begin{bmatrix} cos(\theta_p) & sin(\theta_p) & 0 \\ -sin(\theta_p) & cos(\theta_p) & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \Delta x \\ \Delta y \\ \Delta\theta \end{bmatrix} \tag{1}$$

Donde el vector desplazamiento global resulta en las coordenadas del punto respecto al marco local, ya que el origen del marco local está fijado en el centro del robot.

$$\begin{bmatrix} x_L \\ y_L \\ \theta_L \end{bmatrix} = \begin{bmatrix} cos(\theta_p) & sin(\theta_p) & 0 \\ -sin(\theta_p) & cos(\theta_p) & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \Delta x \\ \Delta y \\ \Delta\theta \end{bmatrix} \tag{2}$$

De la misma forma, un punto con respecto al marco local del robot puede traducirse a un delta/vector desplazamiento desde el centro del robot hasta ese punto en el marco global:

$$\begin{bmatrix} \Delta x \\ \Delta y \\ \Delta\theta \end{bmatrix} = \begin{bmatrix} cos(\theta) & -sin(\theta) & 0 \\ sin(\theta) & cos(\theta) & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_L \\ y_L \\ \theta_L \end{bmatrix} \tag{3}$$

Cinemática Directa e Inversa

Dada la geometría radial del chasis, con 2 ruedas dispuestas en el mismo eje de rotación, es posible hallar la velocidad de avance que lleva el robot y la velocidad angular en base a la velocidad angular de cada rueda.

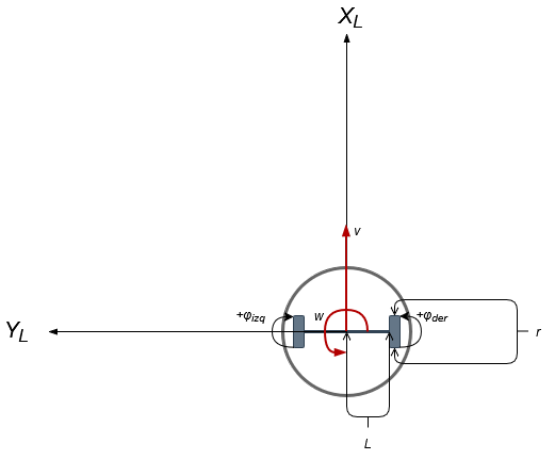


Figura 10: Marco Local de referencia del robot.

Sean ω_{der} y ω_{izq} las velocidades angulares de cada rueda, es posile saber el desplazamiento lineal que aporta cada rueda multiplicando su velocidad angular por su radio r.

Podemos hallar la velocidad de avance del robot sumando los aportes por separado de cada rueda.

La velocidad angular del chasis, podemos hallarla nuevamente tomando por separado los aportes de cada rueda. Para esto se analiza cuánto giraría el robot si una de las ruedas estuviera fija y la otra rotara. En el caso de que tomaramos de pivot la rueda izquierda, la rotación aportada por la rueda derecha sería $\omega_{der} r/2l$.

Queda entonces la velocidad de avance y la velocidad angular del robot expresadas en base a la velocidad angular de cada rueda:

$$\begin{cases} v = \frac{\omega_{der} r + \omega_{izq} r}{2} \\ w = \frac{\omega_{der} r - \omega_{izq} r}{2l} \end{cases}$$

$$\begin{bmatrix} v \\ w \end{bmatrix} = \begin{bmatrix} \frac{r}{2} & \frac{r}{2} \\ \frac{r}{2l} & \frac{-r}{2l} \end{bmatrix} \begin{bmatrix} \omega_{der} \\ \omega_{izq} \end{bmatrix} \tag{4}$$

En base a esto es posible hacer el camino inverso; cuánto debería girar cada rueda para que el robot tenga una velocidad de avance v y una velocidad angular w Despejando $[\omega_{der}, \omega_{izq}]^T$ queda:

$$\begin{bmatrix} \omega_{der} \\ \omega_{izq} \end{bmatrix} = \begin{bmatrix} \frac{r}{2} & \frac{r}{2} \\ \frac{r}{2l} & \frac{-r}{2l} \end{bmatrix}^{-1} \begin{bmatrix} v \\ w \end{bmatrix}$$

$$\begin{bmatrix} \omega_{der} \\ \omega_{izq} \end{bmatrix} = \begin{bmatrix} \frac{1}{r} & \frac{l}{r} \\ \frac{1}{r} & \frac{-l}{r} \end{bmatrix} \begin{bmatrix} v \\ w \end{bmatrix} \tag{5}$$

5.2.2. Algoritmos de Control Cinemático para Navegación

El subsistema de control se plantea como un control configurable de 3 modos.

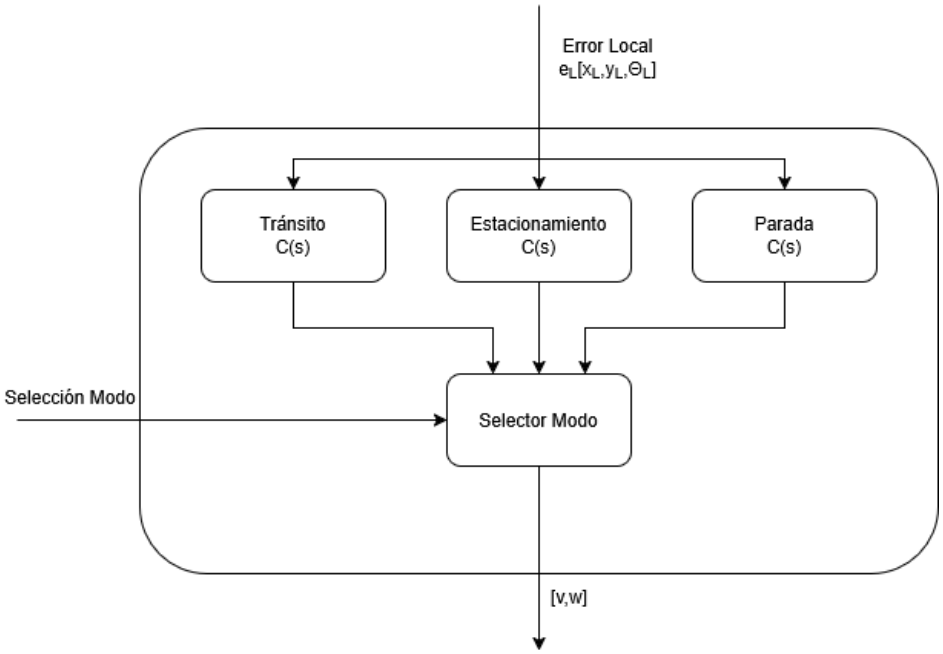


Figura 11: Planteo Modos de Control.

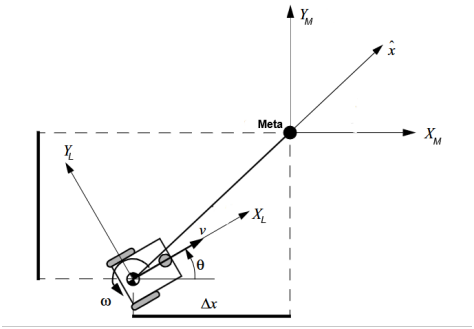


Figura 12: Planteo control.

- **Modo Tránsito:** Implementa un control de seguimiento de trayectoria donde no importa la orientación del chasis sino seguir los puntos de referencia.
- **Modo Estacionamiento:** Implementa el control polar de Siegwart (libro). Activando este control a partir de una distancia minima al punto final es posible garantizar una convergencia al punto final en coordenadas $[x, y, \theta]$.
- **Modo Parada:** Modo trivial de control que fuerza las salidas del control a 0. Este modo se elije al cumplirse el criterio de llegada ($\rho_{dist} < \rho_{tolerancia}$ y $\theta_{dist} < \theta_{tolerancia}$).

Control: Modo Parada

Este control es el más sencillo de todos. Consiste en implementar un control trivial que fuerza la salida $[v, w]$ a $[0, 0]$. El control queda:

$$\begin{bmatrix} v \\ w \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} x_e \\ y_e \\ \theta_e \end{bmatrix}$$

$$\begin{bmatrix} v \\ w \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Análisis del Control Polar de Siegwart (Punto a Punto)
Siegwart (Autonomous Mobile Robots) propone un control polar, donde el vector velocidad desde la posición actual del robot hasta la meta se expresa en coordenadas polares.

Puede verse que por la geometría del chasis, el robot únicamente tiene 2 coordenadas de movimiento: x_l y θ, por lo tanto una transformación de Cartesianas a Polares simplifica el control. Se definen 3 variables polares $[\rho, \alpha, \beta]^T$ equivalentes a las cartesianas $[x, y, \theta]^T$.

- ρ **(Distancia):** Representa la distancia entre el centro del robot y la meta. Reemplaza la magnitud espacial de (x, y).
- α **(Ángulo de apuntamiento):** Es el ángulo entre la dirección a la que apunta la trompa del robot (X_L) y la línea recta que une al centro del robot con la meta $(\hat{x})$.
- β **(Ángulo de alineación final):** Es el ángulo entre la línea recta que une el centro del robot con la meta $(\hat{x})$ y el eje de orientación final deseado (X_M).

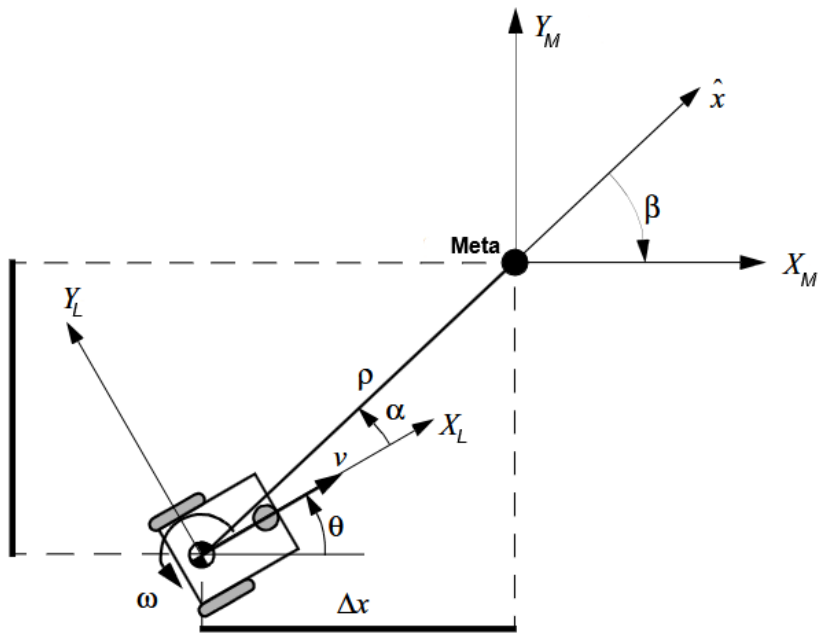


Figura 13: Variables control polar.

El error de posición del robot con respecto a la meta puede traducirse de las coordenadas cartesianas $[\Delta x, \Delta y, \theta_M]$ a las polares como:

- $\rho = \sqrt{\Delta x^2 + \Delta y^2}$
- $\alpha = atan2(\Delta y, \Delta x) - \theta$
- $\beta = -\theta - \alpha + \theta_M = \theta_M - atan2(\Delta y, \Delta x)$

Se propone entonces una ley de control proporcional:

$$\begin{bmatrix} v \\ w \end{bmatrix} = \begin{bmatrix} k_\rho & 0 & 0 \\ 0 & k_\alpha & k_\beta \end{bmatrix} \begin{bmatrix} \rho \\ \alpha \\ \beta \end{bmatrix}$$

v hace que el robot avance mientras más lejos esté y ω ajusta el giro para apuntar a la meta (α) y quedar alineado al llegar (β).

El desarrollo formal del modelo cinemático diferencial en coordenadas polares y su análisis se encuentran en detalle en Siegwart [**?**]. De dicho análisis se derivan las condiciones necesarias para las ganancias del controlador que aseguran que el sistema no oscile y sea estable, las ganancias han de respetar las siguientes condiciones:

$$k_\rho > 0; k_\beta < 0; k_\alpha - k_\rho > 0$$

Análisis del Control para algoritmo Pure Pursuit (Seguimiento de trayectorias)

5.2.3. Análisis Discreto e Implementación Algorítmica

Frecuencia de funcionamiento

Desde una perspectiva física el lazo cinemático no puede recalcular el rumbo más rápido de lo que las ruedas cambian de velocidad. Si la cinemática intetara actualizar las consignas de velocidad $[v, w]$ a la misma frecuencia con la que trabaja la tracción, el control cinemático estaría respondiendo a estados transitorios e incompletos de los motores. Ejecutar el lazo de cinemática entre 5 y 10 veces más lento que el lazo de tracción asegura que, cada vez que la cinemática calcula una nueva consigna de velocidad para cada rueda, los motores ya alcanzaron régimen permanente de la consigna dada en el paso anterior. Queda entonces definida la frecuencia de funcionamiento del lazo de cinemática como:

$$f_{cinemática} = \frac{f_{traccion}}{10}$$

$$\Delta t_{cinemática} = \Delta t_{traccion} 10$$

5.3. Bloque de Mapeo y Estimación de Posición

5.3.1. Análisis Experimental de Deriva Odométrica en theta

5.3.2. Corrección de orientación mediante IMU

5.3.3. Modelado del sensor TOF y construcción de la grilla de ocupación

5.4. Bloque de Navegación Autónoma

5.4.1. Estrategia de exploración y selección de metas

5.4.2. Algoritmo de evasión de obstáculos

5.5. Integración de Lazos y Arquitectura de Control Global

5.5.1. Diagrame unificado del sistema de control

5.5.2. Máquina de estados y lógica de selección de modos

6. Diseño de Firmware

El diseño del firmware para un sistema embebido robótico representa un desafío crítico: se deben ejecutar de manera concurrente tareas de control determinista en tiempo real junto con algoritmos pesados de procesamiento espacial y servicios no deterministas de comunicación inalámbrica.

Para evitar los cuellos de botella característicos de las arquitecturas tradicionales basadas en bucles secuenciales infinitos (*Super-Loops*), se ha seleccionado una **Arquitectura Jerárquica en Capas** gobernada por el kernel de tiempo real **FreeRTOS** sobre la plataforma EDU-CIAA.

6.1. Arquitectura propuesta y criterios de diseño

El firmware se diseñará para integrar la percepción, la estimación de la pose, la construcción del mapa y el control del rover diferencial. La arquitectura será portable entre plataformas: el desarrollo y la validación funcional se realizarán primero sobre ESP32 ya que no contamos con una EDU-CIAA en nuestras casas para desarrollar y probar software de aplicación; luego se portará a EDU-CIAA reutilizando la aplicación y los drivers compartidos, y reemplazando la capa de adaptación al hardware.

Se adopta una organización en tres capas para separar las operaciones propias de cada placa de los algoritmos del robot. Esta separación permite probar los módulos por partes, mantener una única implementación de la lógica del rover y evitar que las decisiones de una placa condicionen el código común. La arquitectura de ejecución se organizará en tareas periódicas controladas por tiempo y planificadas por FreeRTOS, la implementación y caracteristicas de este todavia no se decidieron ya que primero optamos por validar los datos del hardware e integrarlo con una simple versión del scheduler.

El modelo elegido prioriza interfaces estables entre capas, unidades físicas comunes y un main compartido. El período de muestreo y las prioridades se ajustarán mediante mediciones de carga y latencia cuando se integren los encoders, la construcción del mapa y el control de motores.

6.2. Las Tres Capas

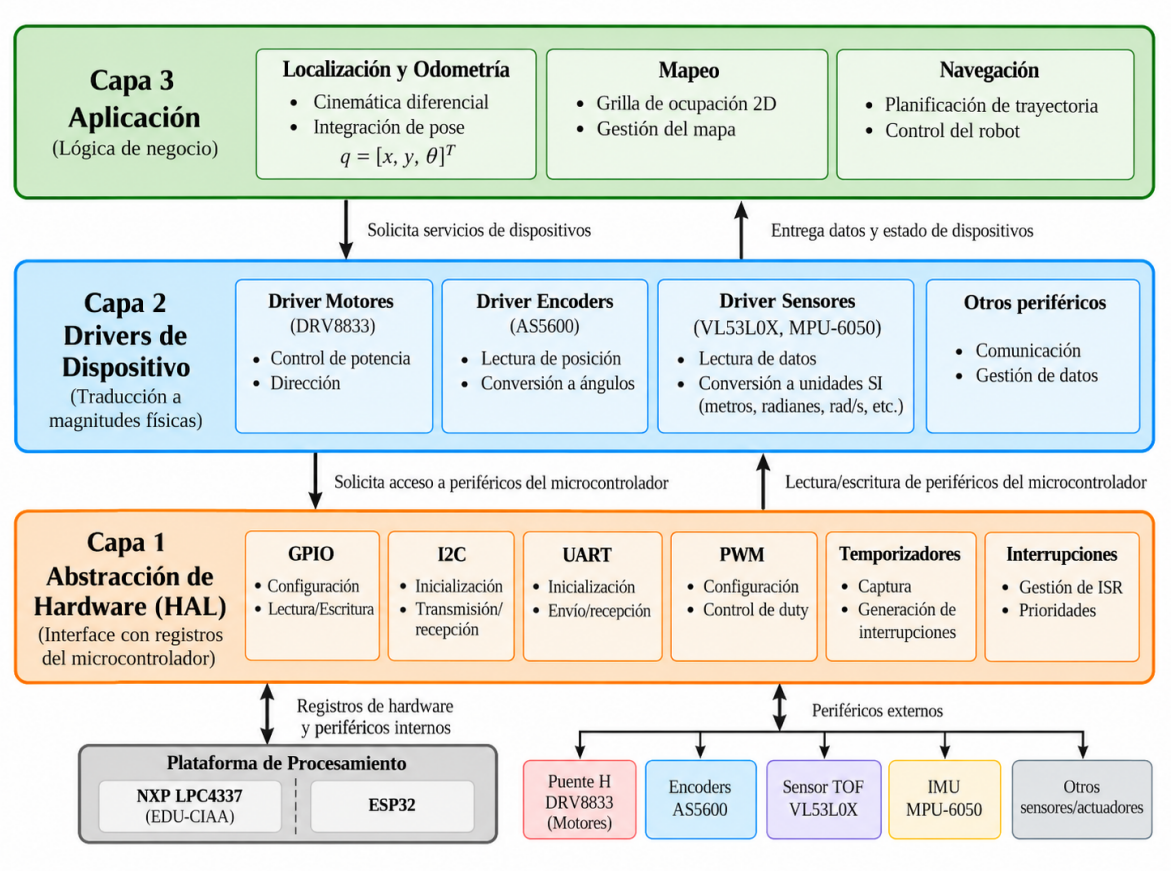


Figura 14: Estructura jerárquica de la arquitectura en capas para el firmware de Arturito.

Capa 1 — HAL (Hardware Abstraction Layer). Será una interfaz común y tendra una implementación por placa. Su objetivo principal es abstraer el acceso a los periféricos y recursos de bajo nivel del microcontrolador, tales como GPIO, I2C, UART, PWM, temporizadores e interrupciones, evitando que las capas superiores dependan directamente de registros o características especificas de una arquitectura.

Cada plataforma tendrá su propia implementación de estas funcionalidades. Por ejemplo, una implementación para la EDU-CIAA podrá configurar directamente registros del LPC4337, mientras que otra implementación podrá utilizar funciones provistas por sAPI o LPCOpen. De manera equivalente, una implementación para ESP32 podrá utilizar los drivers proporcionados por ESP-IDF.

Por lo tanto, la existencia de la HAL no implica que todos los periféricos deban implementarse desde cero mediante acceso directo a registros. Si la arquitectura dispone de una biblioteca o driver que ya implementa dicha funcionalidad, la función correspondiente de la HAL podrá encapsular esa implementación. De esta manera, las capas superiores mantienen siempre la misma interfaz independientemente de cómo se resuelva internamente cada operación.

La HAL no realizará tratamiento de las mediciones ni algoritmos propios del funcionamiento del rover. Sin embargo, en aquellos casos en que una biblioteca específica de una plataforma ya proporcione funciones de mayor nivel para acceder

a un dispositivo, la HAL podrá encapsular dichas funciones con el fin de mantener una interfaz uniforme entre arquitecturas. En estos casos terminaria actuando como una HAL de perifericos. El procesamiento común de las mediciones continuará perteneciendo a los drivers de dispositivos. La HAL tampoco realizará integración de pose, mapeo ni navegación.

Ejemplos de funciones HAL:
- `hal_gpio_config()`
- `hal_gpio_write()`
- `hal_i2c_init()`
- `hal_i2c_read()`
- `hal_pwm_set_duty()`
- `hal_imu_get_gyro_z_rads()` representa el caso en que la capa se comporta como HAL de perifericos. Si la plataforma dispone de una biblioteca específica para un periferico, podría encapsular directamente la función equivalente de dicha biblioteca. De esta forma, las capas superiores utilizan la misma interfaz independientemente de la implementación disponible en cada arquitectura.

Capa 2 — Drivers de dispositivos. Interpretará las señales físicas de cada dispositivo y entregará datos listos para la aplicación. Incluirá lectura, calibración, corrección y validación de datos de los sensores y podrá enviar comandos de generación de señales de control para el DRV8833. Sus interfaces no dependerán de Arduino ni de sAPI.

Ejemplos de funciones de drivers:
- `driver_mpu6050_get_gyro_z_rads()`
- `driver_vl53l0x_get_distance_m()`
- `driver_as5600_get_angle_rad()`
- `driver_motor_set_pwm()`
- `driver_motor_set_direction()`

Capa 3 — Aplicación. Contendrá la lógica propia del rover: adquisición y sincronización de muestras, cinemática diferencial y estimación de pose, actualización de la grilla de ocupación, selección de consignas según el modo y distribución de datos a la navegación y las comunicaciones. Acá vivirán los algoritmos de estimación de pose, mapeo y navegación.

Ejemplos de funciones de aplicación:
- `odometry_update()`
- `pose_estimator_update()`
- `occupancy_grid_update()`
- `navigation_update()`

Infraestructura transversal — FreeRTOS. FreeRTOS no se considerará una capa adicional de la arquitectura, sino una infraestructura transversal utilizada para organizar la ejecución concurrente del sistema. El kernel se encargará de la planificación de las tareas, sus prioridades y temporización, además de proporcionar mecanismos de comunicación y sincronización.

La lógica de aplicación podrá dividirse en tareas periódicas o activadas por eventos,

por ejemplo para adquisición de sensores, control de velocidad, estimación de pose, actualización del mapa, navegación y comunicaciones. Cuando resulte necesario, los drivers y la HAL también podrán utilizar mecanismos proporcionados por FreeRTOS para sincronizar accesos a recursos compartidos.

6.3. Software de PC e interfaz de usuario prevista

El alcance para el programa de PC es visualizar la pose del rover y el mapa actualizado. Para transmitir pose y mapa sin cable se prevé incorporar Wi-Fi o Bluetooth. La elección entre ambos protocolos queda pendiente. Además desde allí se podran mandar órdenes START/STOP y, como objetivos secundarios, teleoperación y planificación de rutas; su incorporación como controles de esta interfaz queda por definir.



Figura 15: Interfaz para ver el mapeo desde el punto de vista del usuario.

Desde esta vista se puede imaginar el siguiente ciclo:

Inicio → conexión con el rover → espera de datos → visualización continua de pose y mapa → pérdida de enlace: indicar desconexión / conexión restablecida: reanudar visualización

Si se pierde el enlace durante la operación, el firmware deberá aplicar la parada segura y la interfaz indicará la desconexión.

6.4. Bibliotecas y herramientas externas previstas

- **ESP32:** framework Arduino y su integración con FreeRTOS; `Wire` para I2C; biblioteca Adafruit `VL53L0X` para el sensor ToF. Para el transporte inalámbrico se utilizará la capacidad disponible en ESP32 una vez elegido el protocolo de enlace. Para la compilación, carga y estructura general del sistema se utiliza PlatformIO.
- **EDU-CIAA:** LPCOpen y SAPI para la placa y sus periféricos, más el kernel FreeRTOS configurado en el proyecto. La HAL de algunos modulos estan implementados (por ejemplo: MPU60x0, el cual brinda acceso simplificado al sensor MPU6050) mientras que hay sensores como el ToF que no tienen esta libreria especifica y al cual le tendremos que desarrollar la implementación de acceso I2C por registros.
- **Workspace:** Se logró integrar ambos entornos de desarrollo en un mismo wokspace para VScode.

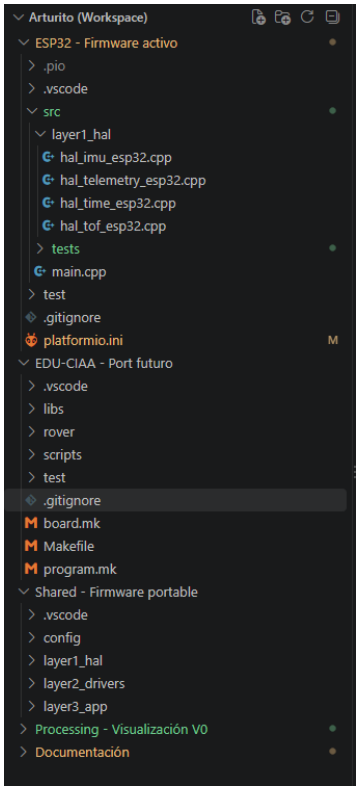


Figura 16: Estrucura del Workspace.

`docs/` Documentos del equipo.

`shared/` Contratos HAL, drivers y logica de aplicación que no deben depender de Arduino ni de sAPI.

`esp32/` Firmware PlatformIO para ESP32, sus HAL de placa y programas de diagnostico independientes.

`edu-ciaa/` Copia de estructura del entorno de 'Embedded IDE'. Make, LPCOpen, sAPI, todo lo necesario para la compilación y carga sobre EDU-CIAA. Adapta la misma aplicación y drivers a su arquitectura.

`processing/` Por ahora esta carpeta es nuestra "PC externa". Contiene sketches que reciben datos para visualizar.

6.5. Versiones y ensayos realizado

6.5.1. Pruebas IMU y ToF

Esta versión ya cuenta con la estructura base del workspace explicado anteriormente pero solo realiza el siguiente ciclo de percepción:

1. Inicializa la IMU MPU6050 y el sensor de distancia VL53L0X.
2. Lee la velocidad angular del eje Z de la IMU y una distancia del ToF.
3. Corrige e integra la velocidad angular para estimar el angulo θ.
4. Acepta distancias entre 0.05 m y 1.50 m; el resto se representa como -1 (medida invalida o sensor no disponible).
5. Publica una linea CSV con θ en radianes y distancia en metros.
6. Processing (programa graficador) representa la orientación y acumula puntos del ToF mientras el robot gira en el lugar.

No hay todavía odometria XY, encoders, motores, navegación autónoma ni grilla de ocupación. No hay modos de funcionamiento por comandados ni comunicación inalámbrica.

El firmware fue cargado tanto en ESP32 como en EDU-CIAA y validado con Processing: se comprobó la recepción de las muestras serie y la visualización de orientación y distancia.

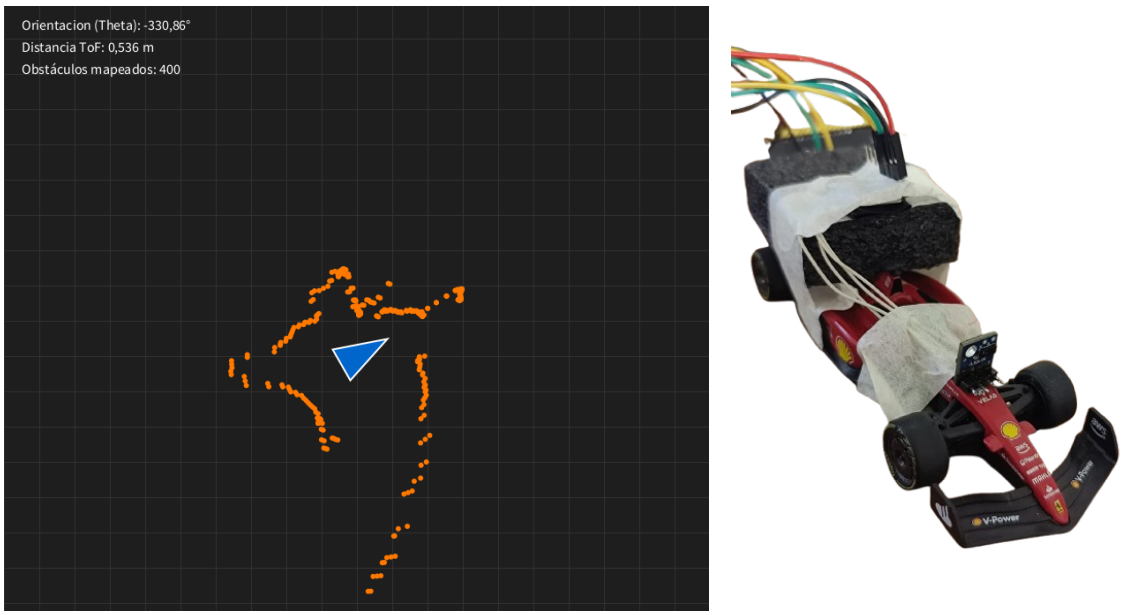


Figura 17: Visualización en Processing (izquierda) y sensores montados en un auto a control remoto (derecha). Link para ver funcionamiento en video.

Cómo llegamos a este punto:

El Firmware es en realidad una versión final de un proceso de testeo de los sensores en ambas arquitecturas, de un proceso de interpretación de datos, de armado de workspace, de ejecución del programa con FreeRTOS, etc.

Algunos cosas interesantes de mensionar:

Angulo entregado por capa 1 vs Angulo provisto por capa 2: La capa 2 espera que el robot quede quieto, promedia 1000 lecturas separadas por 2 ms para estimar el offset, y reinicia el estado de integración, luego resta el offset, elimina velocidades dentro de $\pm 1 \circ /s$, calcula un promedio trapezoidal entre muestras y acumula el angulo con el tiempo transcurrido.

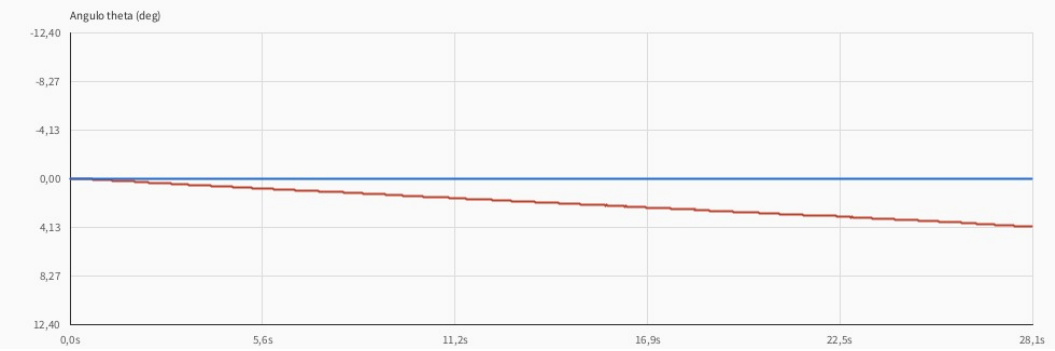


Figura 18: Obtención del angulo crudo vs. procesado por capa 2.

Se ve que a tan solo 30s de inicializado el rover el angulo crudo difiere en 5° del angulo real (el rover estaba quieto).

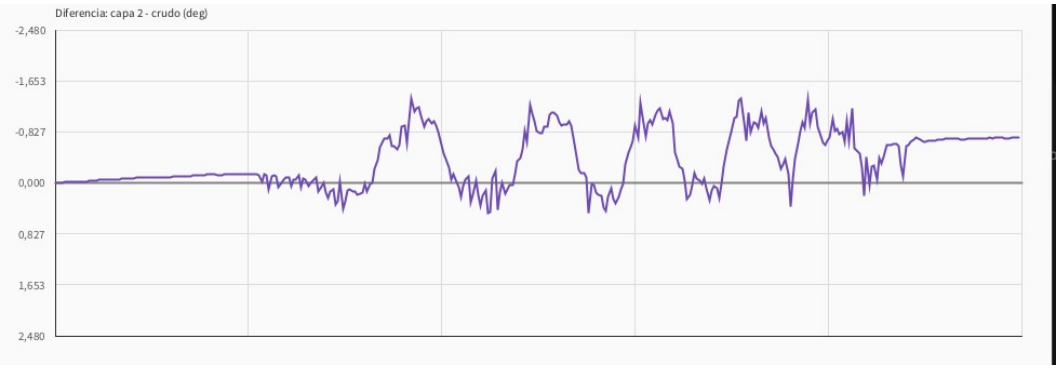


Figura 19: Angulo procesado - Angulo crudo

Acá se puede ver que si se mueve el robot cambia la relacion entre una señal y la otra. Si el procesado no fuese necesario entonces este grafico deberia ser casi siempre 0 o aunque sea aproximarse a 0.

Error en la librería de MPU para EDU-CIAA: como ya mencionamos, existe una librería que otorga recursos para manejar el giroscopio y acelerometro MPU6050 dentro de sAPI. Durante las pruebas de integración en hardware de se identificó un error que hacia que el sensor no responda.

El procedimiento de arranque de la libreria implementaba una verificación de integridad mediante la lectura del registro interno WHO_AM_I (dirección 0x75),

cuyo valor de retorno esperado según la hoja de datos estándar del fabricante es 0x68. Sin embargo, la función de inicialización abortaba de forma consistente retornando un código de falla (-5), impidiendo la ejecución del resto del sistema.

Tras analizar el bus de comunicación I2C mediante rutinas de escaneo, se constató que la variante física del módulo utilizado responde con el identificador 0x72 no 0x68. Entendimos que este comportamiento es habitual en hardware genéricos y para solucionar esta incompatibilidad se reestructuró la validación en el driver. La lógica fue modificada para admitir de forma flexible tanto el identificador nativo (0x68) como el valor identificado en el banco de pruebas (0x72).

6.5.2. Estructura de compilación y ejecución de aplicaciones

A medida que avanzó el desarrollo del firmware, se incorporaron distintas aplicaciones de prueba para verificar individualmente los sensores, actuadores y módulos del sistema, antes de integrarlos en el funcionamiento general del rover.

Uno de los objetivos fue mantener la compatibilidad con ambas arquitecturas, ESP32 y EDU-CIAA, evitando desarrollar dos versiones independientes de una misma aplicación. Para ello, se implementó un mecanismo común de inicialización y ejecución, adaptado a los requisitos particulares de cada plataforma.

En el caso del ESP32, utilizando el framework Arduino, la ejecución requiere las funciones `setup()` y `loop()`, destinadas a la inicialización y ejecución repetitiva del programa, respectivamente. Por otro lado, la EDU-CIAA utiliza una función `main()`, desde la cual se inicializan los recursos de la placa y se controla el ciclo principal de ejecución.

Para unificar ambos mecanismos se definieron dos funciones comunes:

- `app_init()`: realiza la inicialización de los recursos y módulos necesarios para la aplicación seleccionada.
- `app_update()`: ejecuta una iteración del funcionamiento de dicha aplicación.

Cada arquitectura dispone de un archivo de entrada propio (`entry_esp32.cpp` y `entry_educiaa.c`), encargado de inicializar los recursos generales de la plataforma y llamar a estas dos funciones. En ESP32, las llamadas se realizan desde `setup()` y `loop()`, mientras que en EDU-CIAA se efectúan desde `main()`, utilizando un ciclo `while(1)`.

De esta manera, las aplicaciones de capa 3 no necesitan conocer el mecanismo de arranque de la placa donde se ejecutan. Las diferencias de hardware continúan resolviéndose mediante las implementaciones específicas de la HAL, conservando la separación por capas definida anteriormente.

Selección y compilación de aplicaciones: Cada prueba se implementa como una aplicación independiente que define las funciones `app_init()` y `app_update()`. Para ejecutarla, se seleccionan durante la compilación los archivos correspondientes a dicha aplicación, junto con los drivers y las implementaciones de HAL que necesita.

El procedimiento de selección depende del entorno de desarrollo:

ESP32: Se utiliza PlatformIO, donde el archivo `platformio.ini` contiene distintos entornos de compilación. Cada entorno especifica los archivos fuente y las dependencias necesarias mediante `build_src_filter`, además de heredar una configuración común de la placa. Para ejecutar una prueba diferente, basta con seleccionar su entorno correspondiente.

EDU-CIAA: Se utiliza el sistema de compilación basado en Make. El archivo `program.mk` define los modos de aplicación disponibles mediante la variable `APP_MODE`, mientras que `rover/config.mk` selecciona los archivos fuente y las dependencias correspondientes. El modo puede indicarse directamente al ejecutar el comando `make`, sin necesidad de modificar el `Makefile` principal.

Esta organización permite realizar pruebas individuales de la IMU, el sensor ToF, los encoders, los motores y otros módulos, así como desarrollar ensayos que integren varios componentes.

La principal ventaja es que una misma aplicación de prueba puede reutilizarse en ambas arquitecturas, cambiando únicamente el entorno de compilación y las implementaciones de hardware utilizadas. Además, permite incorporar nuevas pruebas sin modificar los puntos de entrada ni alterar las aplicaciones existentes, facilitando el desarrollo incremental y la detección de errores durante la integración.

6.5.3. Sistema de telemetría

Para facilitar las pruebas y el diagnóstico de los distintos módulos, se desarrolló un sistema de telemetría que permite transmitir información desde el rover hacia una computadora externa.

Se buscó que las aplicaciones pudieran enviar información independientemente de la arquitectura utilizada, manteniendo una interfaz común y delegando las particularidades de comunicación a las implementaciones de HAL de cada placa.

El sistema utiliza el ESP32 como encargado de establecer la comunicación inalámbrica mediante Wi-Fi. Actualmente funciona en modo Access Point (AP), creando una red propia denominada `Arturito`, a la cual puede conectarse una computadora sin necesidad de contar con un router externo. Sobre esta conexión se utiliza un servidor TCP, que transmite los mensajes de telemetría a través del puerto 3333.

La implementación presenta dos modalidades según la arquitectura que ejecuta la aplicación:

Aplicación ejecutada en ESP32: La aplicación utiliza la interfaz de telemetría compartida, cuya implementación de HAL envía los mensajes directamente a través del módulo Wi-Fi del ESP32. De esta manera, un único firmware realiza tanto el procesamiento de la aplicación como la transmisión de sus resultados.

Aplicación ejecutada en EDU-CIAA: En este caso, la EDU-CIAA no dispone de conectividad Wi-Fi propia, por lo que utiliza una comunicación UART para transmitir los mensajes hacia el ESP32.

Para ello, se carga en el ESP32 un firmware independiente que funciona como puente UART–Wi-Fi. Este programa recibe los mensajes enviados por EDU-CIAA

y los retransmite mediante la conexión TCP, sin realizar el procesamiento de los datos recibidos.

En ambos casos, las aplicaciones utilizan las mismas funciones de telemetría, sin necesidad de conocer el medio físico de transmisión. Esto permite mantener compartido el código de las pruebas y modificar la arquitectura utilizada sin alterar su funcionamiento general.

La transmisión se realiza mediante mensajes de texto delimitados por saltos de línea, lo que facilita su recepción e interpretación desde una computadora.

Se desarrolló una prueba específica de telemetría que transmite mensajes periódicos, con el objetivo de verificar la comunicación y el funcionamiento de ambas modalidades.

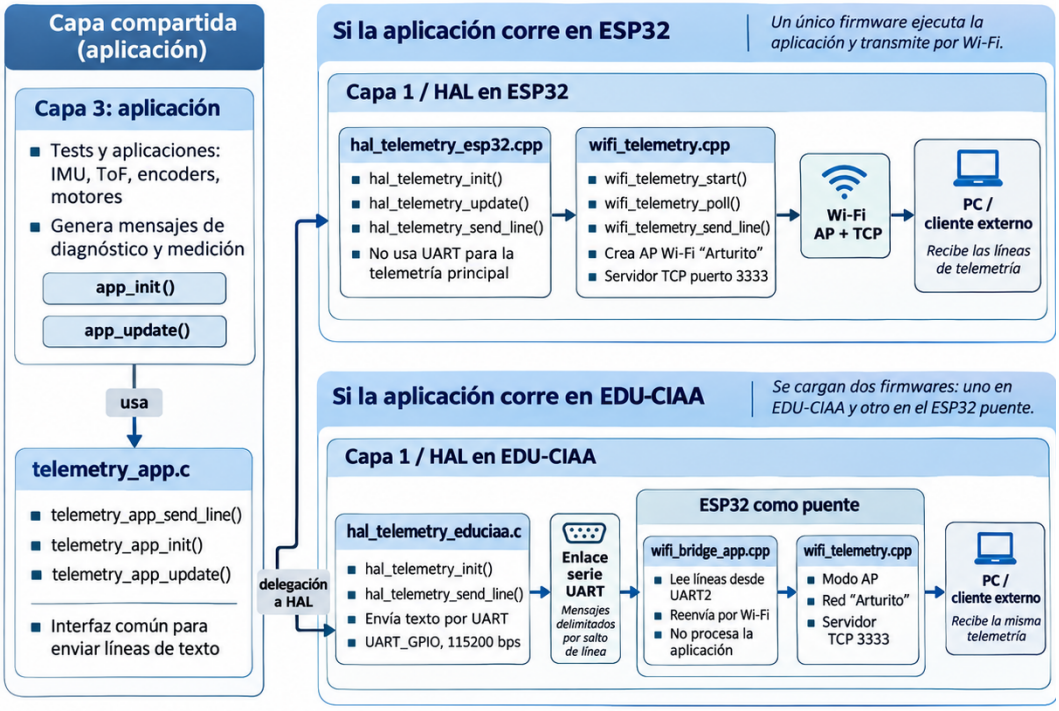


Figura 20: Telemetría para ambas arquitecturas

6.5.4. Pruebas de sensores y actuadores

Una vez establecida la estructura común de ejecución de aplicaciones y el sistema de telemetría, se desarrollaron distintas pruebas para verificar el funcionamiento de los sensores y actuadores del rover.

Estas pruebas permitieron evaluar los módulos de forma independiente antes de avanzar hacia su integración. En todos los casos se mantuvo la arquitectura por capas, utilizando los mismos drivers y lógica de aplicación para ambas plataformas, con las correspondientes implementaciones de HAL.

Prueba de IMU (MPU6050): Se desarrolló una aplicación que inicializa y calibra el sensor, solicitando que el rover permanezca quieto durante este procedimiento.

Una vez finalizada la calibración, se realizan lecturas periódicas de la velocidad angular para estimar el ángulo de orientación alrededor del eje Z. El ángulo

calculado es transmitido mediante telemetría, permitiendo observar su comportamiento mientras se gira físicamente el sensor.

Esta prueba permite verificar tanto la comunicación con la IMU como el funcionamiento del procesamiento de orientación implementado en las capas superiores.

Prueba de distancia (VL53L0X): Se implementó una aplicación para verificar las mediciones del sensor ToF mediante comunicación I2C.

El sensor se inicializa utilizando el perfil de medición High Speed y se realizan lecturas periódicas de distancia. Para cada medición se transmiten dos valores: la distancia obtenida directamente del sensor y la distancia corregida mediante la función de calibración implementada.

De esta manera, es posible comparar ambas mediciones y evaluar el comportamiento de la corrección aplicada. La prueba también contempla la detección de mediciones inválidas y errores de comunicación.

Pruebas de encoders (AS5600): Se desarrollaron pruebas para verificar la comunicación I2C con los encoders magnéticos instalados en las ruedas del rover.

Inicialmente se realizaron lecturas de la posición angular absoluta de cada encoder, girando manualmente las ruedas y transmitiendo los valores obtenidos.

Posteriormente, estas lecturas se utilizaron para calcular la velocidad angular a partir de la variación de posición y el tiempo transcurrido entre muestras, contemplando el salto de ángulo que se produce al completar una revolución.

Pruebas de motores (DRV8833): Se implementaron aplicaciones para comprobar el accionamiento de los motores mediante señales PWM generadas a través de la HAL correspondiente.

Se ensayaron diferentes secuencias de movimiento, incluyendo avance, retroceso y giros sobre el propio eje. También se desarrollaron pruebas de trayectorias predefinidas, utilizando tiempos de actuación y diferentes valores de PWM.

Estas primeras pruebas se realizaron en lazo abierto, sin utilizar realimentación de los encoders para corregir el movimiento.

Integración de motores y encoders: Finalmente, se combinaron las pruebas anteriores para accionar ambos motores y obtener simultáneamente información de los encoders.

Se implementaron secuencias con distintos sentidos de giro y valores de PWM, mientras se calculaba y transmitía la velocidad angular de las ruedas en RPM.

Esta integración permite relacionar la señal de control aplicada a los motores con la velocidad medida, constituyendo una primera base experimental para el desarrollo posterior del control de velocidad.

Los resultados y mensajes de diagnóstico de estas pruebas se transmitieron mediante el sistema de telemetría, permitiendo observar el comportamiento de los distintos módulos desde una computadora externa.

7. Cronograma Preliminar

Cronograma (Google Sheets)

En el siguiente enlace podrá acceder al cronograma preliminar detallado:

Enlace: Ver Planilla de Cronograma en Google Sheets

8. División de Tareas del Grupo

División de tareas (Google Sheets)

En el siguiente enlace podrá acceder a una plantilla con los módulos y requerimientos a realizar por integrante:

Enlace: Ver Planilla de División de tareas en Google Sheets

9. Bibliografía

Referencias

[1] R. Siegwart, I. R. Nourbakhsh y D. Scaramuzza, *Introduction to Autonomous Mobile Robots*, 2.a ed. Cambridge, MA, USA: MIT Press, 2011, cap. 3, pp. 89–108. URL: https://www.ucg.ac.me/skladiste/blog_13268/objava_56689/fajlovi/Introduction%20to%20Autonomous%20Mobile%20Robots%20book.pdf.

[2] S. Thrun, W. Burgard y D. Fox, *Probabilistic Robotics*. Cambridge, MA, USA: MIT Press, 2005, cap. 5, 8, 9, 10. URL: https://docs.ufpr.br/~danielsantos/ProbabilisticRobotics.pdf.

[3] Homemade LIDAR sensor with Arduino & Processing, URL: https://www.youtube.com/watch?v=fQ2iB7qkrUg.

[4] How to Make an Autonomous Mapping Robot Using SLAM, URL: https://www.youtube.com/watch?v=xqjVTE7QvOg.

[5] Autonomous Navigation playlist by MATLAB, URL: https://www.youtube.com/watch?v=Fw8JQ5Q-ZwU&list=PLn8PRpmsu08rLRGrnF-S6TyGrmcA2X7kg.

[6] Texas Instruments, *DRV8833 Dual H-Bridge Motor Driver*, hoja de datos.

[7] Arduino, *Arduino Nano*, hoja de datos.

[8] Microcontrollers Lab, *VL53L0X LIDAR Distance Sensor Interfacing with Arduino*, URL: https://microcontrollerslab.com/vl53l0x-module-interfacing-arduino/.

[9] TinyTronics, *AS5600 Magnetic Angle Sensor Encoder Module (AS5600MOD)*, URL: https://www.tinytronics.nl/en/sensors/magnetic-field/as5600-magnetic-angle-sensor-encoder-module.

[10] Handson Technology, *G12-N20 Geared Mini DC Motor*, hoja de datos, URL: https://www.handsontec.com/dataspecs/motor_fan/GA12-N20.pdf.

[11] Retro-Ciaa,*Lic. Santiago Germino, Biblioteca Digital, Facultad de Ingeniería, Universidad de Buenos Aires* , URL: https://bibliotecadigital.fi.uba.ar/files/original/ecb2999e1e35ec2daea21bd31b3343a1.pdf.

[12] STMicroelectronics, *VL53L0X: Time-of-Flight ranging sensor*, hoja de datos.

[13] ams-OSRAM, *AS5600: 12-Bit Programmable Contactless Potentiometer*, hoja de datos.

[14] Aerosemi, *MT3608 2A Step-Up Converter*, hoja de datos.

[15] Espressif Systems, *ESP32 Series Datasheet*.

[16] Proyecto CIAA, *EDU-CIAA-NXP*, esquemáticos y documentación.

[17] KiCad EDA, URL: https://www.kicad.org/.

[18] MPU 6050 Datasheet, URL: https://www.alldatasheet.com/datasheet-pdf/view/517744/ETC1/MPU-6050.html.

10. Anexo

10.1. Circuito Esquemático

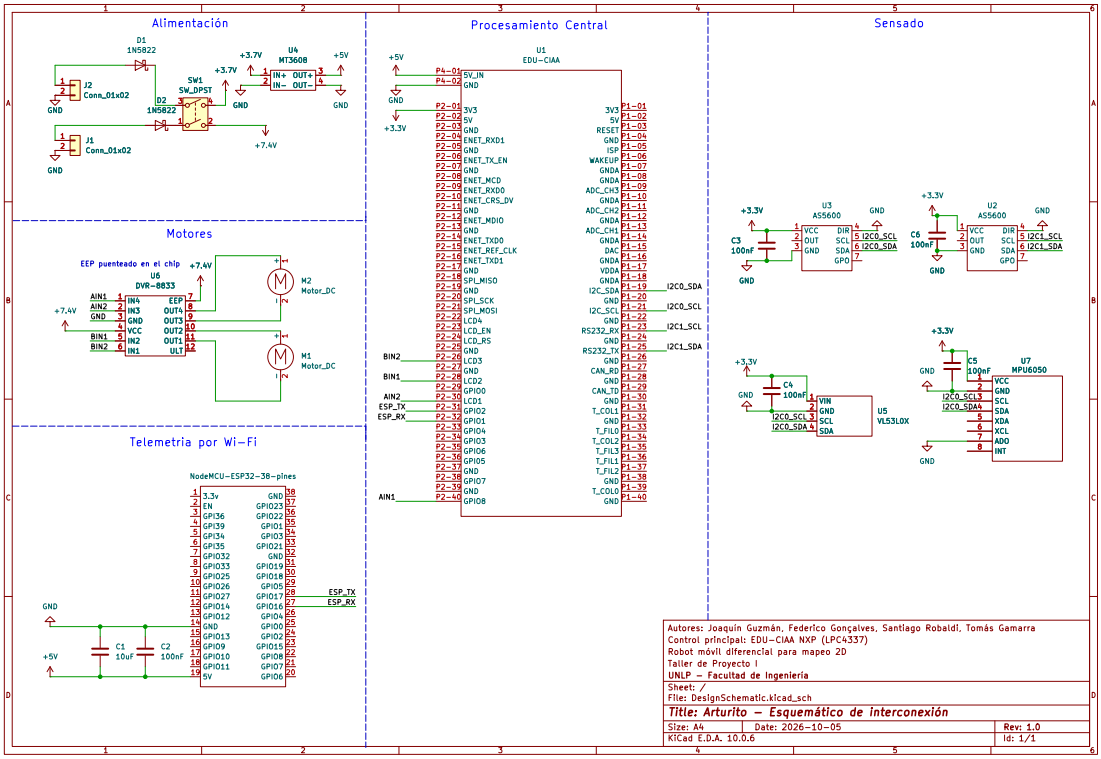


Figura 21: Esquemático completo del sistema.

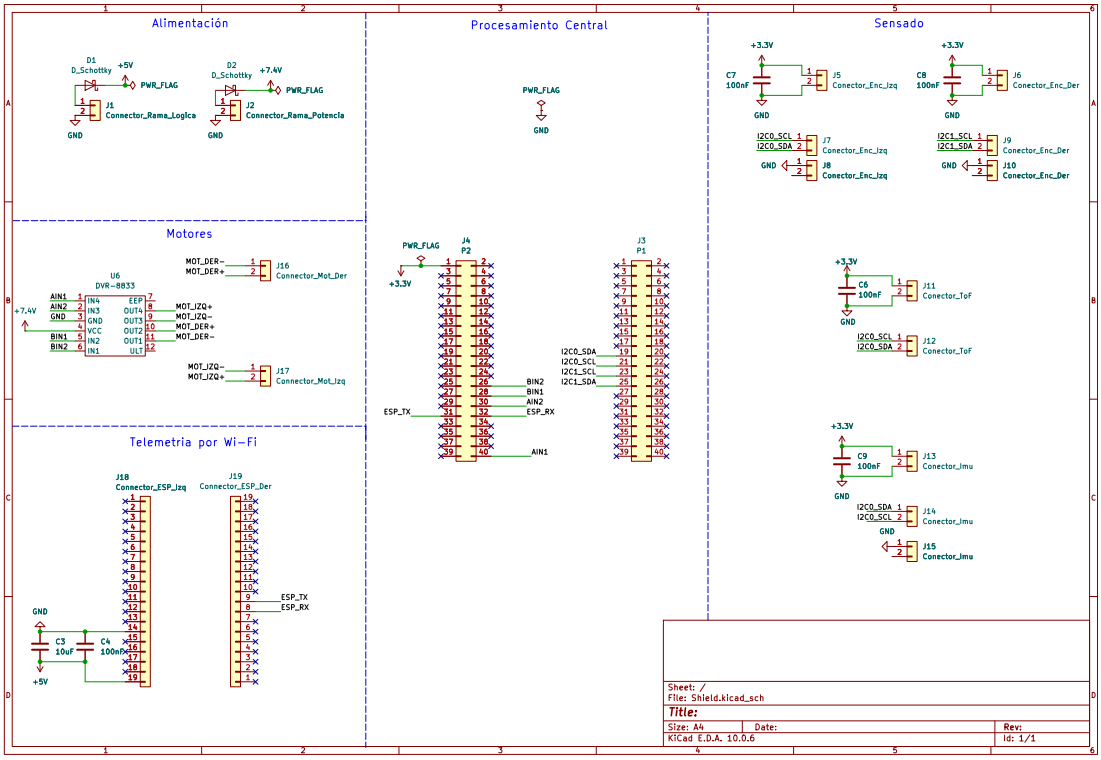


Figura 22: Esquemático completo del poncho del sistema.

10.2. Lista de materiales (BOM)

Tabla 8: Lista de materiales preliminar.

Ítem	Descripción / código de parte	Cant.	Formato
1	Motorreductor GA12-N20 (Handson Technology), 6 V–12 V, 150:1	2	Motor con caja metálica
2	Módulo driver DRV8833	1	Módulo, pines 2,54 mm
3	Módulo elevador MT3608 (5 V, ramal de lógica)	1	Módulo
4	Celda Li-ion 18650, 3,7 V (capacidad a confirmar)	3	Cilíndrica
5	Portapilas 18650: uno de 1 celda y uno de 2 celdas en serie	2	Portapilas
6	Protección de celdas (BMS), no definida	[?]	[?]
7	Cargador de celdas, no definido	[?]	[?]
8	Capacitor electrolítico 10 µF	1	Componente
9	Capacitor monolitico 100 nF (104)	6	Componente
10	ESP32 DevKit v1	1	Módulo
11	IMU MPU6050	1	Módulo
12	Módulo sensor de distancia VL53L0X	1	Módulo
13	Módulo AS5600 (TinyTronics AS5600MOD), con imán diametral incluido	2	Módulo 23 mm×23 mm
14	Ruedas de 44 mm de diámetro	2	Rueda plástica / goma
15	Filamento ABS (para impresión 3D de piezas estructurales)	1 kg	Carrete de filamento
16	Chasis inferior (nivel base para motores, celdas y sensores)	1	Pieza impresa (ABS)
17	Chasis superior modular para prototipado (soporte de *breadboard*)	1	Pieza impresa (ABS)
18	Chasis superior modular (soporte para placa EDU-CIAA)	1	Pieza impresa (ABS)
19	Varilla roscada M3 de 1 m	1	Varilla
21	Tornillos M3	8	Tornillería
22	Tuercas hexagonales M3	16	Tornillería
23	Apoyo trasero: patín impreso en ABS (o *ball caster* de acero)	1	Pieza impresa / Comp.
24	Boton switch 4 pines	1	Componente
25	Diodo Schottky 15A 45V	4	Componente
26	Bornera 2 vías	15	Componente
27	Tira de 40 pines macho	2	Componente
28	Tira de 40 pines hembra	1	Componente

[PENDIENTE: confirmar cantidades y qué materiales ya se poseen, como la EDU-CIAA y los motores del prototipo.]